บทที่ 8 SQL: ภาษาจัดการข้อมูล

Data Manipulation Language

วัตถุประสงค์

- สามารถอธิบายและแสดงข้อมูลจากฐานข้อมูลด้วยคำสั่ง SELECT ได้
- สามารถแสดงข้อมูลตามข้อคำถามด้วยการสร้างคำสั่ง SQL ที่ประกอบด้วย
  - WHERE clause สำหรับแสดงแถวของข้อมูลที่เป็นไปตามเงื่อนไข
  - เรียงลำดับผลลัพธ์จากข้อคำถามด้วยคำสั่ง ORDER BY
  - ใช้งานฟังก์ชันสำหรับข้อมูลเป็นชุด (aggregate function) ด้วยคำสั่ง GROUP BY
  - สร้างข้อคำถามย่อย (subquery)
  - เชื่อมตาราง (join tables)
  - ดำเนินการเกี่ยวกับเซต (UNION, INTERSECTION, EXCEPT)
- สามารถเพิ่มข้อมูลด้วยคำสั่ง INSERT ได้
- สามารถแก้ไขข้อมูลด้วยคำสั่ง UPDATE ได้
- สามารถลบข้อมูลด้วยคำสั่ง DELETE ได้
- สามารถอธิบายประโยชน์และสร้าง Stored Procedures อย่างง่ายได้
- สามารถอธิบายประโยชน์และสร้าง Triggers อย่างง่ายได้

คำสำคัญ: ภาษาที่ใช้ในการจัดการข้อมูล (Data Manipulation Language); SELECT; WHERE; ORDER BY; COUNT; SUM; AVG; MIN/MAX; GROUP BY; IN; HAVING; ANY/SOME; ALL; JOIN; EXISTS; UNION; INTERSECTION; EXCEPT; INSERT; UPDATE; DELETE

บทที่ 8 SQL: ภาษาจัดการข้อมูล

Data Manipulation Language

8.1 บทนำ

SQL ที่อธิบายในบทที่ 7 เป็นกลุ่มของภาษาที่ใช้ในการนิยามโครงสร้างข้อมูลเท่านั้น ยังไม่ได้มีการบรรจุข้อมูลและเรียกใช้ ในบทนี้จะกล่าวถึงการจัดการข้อมูลด้วย SQL ซึ่งในหัวข้อที่ 8.2 อธิบายถึง SQL ที่ใช้ในการแสดงข้อมูลด้วยคำสั่ง SELECT แบบต่างๆ และสำหรับหัวข้อที่ 8.3 อธิบายการใช้งานคำสั่งที่เกี่ยวข้องกับการเพิ่ม/แทรก แก้ไข/ปรับ และลบข้อมูล หัวข้อที่ 8.4 และ 8.5 อธิบายการสร้าง triggers และ stored procedures ซึ่งเป็นการใช้งาน SQL ในการจัดการข้อมูลในขั้นสูงขึ้น โดย triggers คือกลไกที่ใช้ในการดำเนินการกับข้อมูลเมื่อเกิดเหตุการณ์ที่กำหนด และ stored procedures เป็นการเขียนโปรแกรมด้วย SQL

8.2 ข้อคำถาม (Queries)

คำสั่ง SQL ที่ใช้ในการจัดการข้อมูล มีวัตถุประสงค์หลักเพื่อใช้ในการค้นคืนและแสดงข้อมูลตามข้อคำถาม จากฐานข้อมูล ตลอดจนเพิ่มและแก้ไขข้อมูลในฐานข้อมูล การดำเนินการของคำสั่ง SQL นั้นจะประกอบไปด้วยการดำเนินการ selection, projection และ join ตามพีชคณิตเชิงสัมพันธ์ ในคำสั่ง SQL 1 คำสั่ง การใช้งาน SQL กรณีแรกคือการแสดงข้อมูลแบบต่างๆ มีรูปแบบของคำสั่งดังนี้

SELECT	[**DISTINCT** | **ALL**] {* | [column_expression [**AS** new_name]] [,…]}
FROM	table_name [alias] [, …]
[**WHERE**	condition]
[**GROUP BY**	column_list] [**HAVING** condition]
[**ORDER BY**	column_list]

โดย

column_expression คือ ชื่อของคอลัมน์ในตารางข้อมูลซึ่งอาจมีหลายๆ คอลัมน์ก็ได้หรือนิพจน์อื่นๆ ของคอลัมน์ตลอดจนการกำหนดชื่อใหม่ให้กับคอลัมน์

table_name คือ ชื่อของตารางความสัมพันธ์หรือวิวที่จะทำการค้นคืนข้อมูล

สำหรับลำดับในการดำเนินการของคำสั่ง SQL เรียงตามลำดับดังนี้

1. **FROM** เป็นการกำหนดตารางความสัมพันธ์หรือวิวที่เกี่ยวข้องในคำสั่ง
2. **WHERE** เป็นการระบุเงื่อนไขในการแสดงระเบียนข้อมูลที่ตรงตามเงื่อนไข ซึ่งเป็นการดำเนินการ selection และ join ในพีชคณิตเชิงสัมพันธ์
3. **GROUP BY** เป็นการจัดกลุ่มของระเบียนที่มีข้อมูลในคอลัมน์เหมือนกันตามที่ระบุ
4. **HAVING** เป็นการกรองกลุ่มของข้อมูลให้เป็นไปตามเงื่อนไขที่ระบุ
5. **SELECT** เป็นการเลือกแสดงคอลัมน์ ซึ่งเป็นการดำเนินการ projection ในพีชคณิตเชิงสัมพันธ์

6. **ORDER BY** เป็นการกำหนดรูปแบบการแสดงข้อมูลให้เรียงลำดับตามคอลัมน์ที่กำหนด

ลำดับของคำสั่งไม่สามารถเปลี่ยนได้ และต้องเป็นไปตามไวยกรณ์ที่กำหนด SELECT และ FROM เป็นคำที่จะต้องปรากฏอยู่ในคำสั่ง SQL เสมอ ผลลัพธ์ที่ได้จากคำสั่ง SQL คือตารางของข้อมูลผลลัพธ์ หัวข้อต่อไปนี้เป็นการอธิบายและยกตัวอย่างการใช้งานคำสั่ง SQL แบบต่างๆ โดยจะใช้สคีมาตัวอย่างของฐานข้อมูลมหาวิทยาลัย (Ramakrishnan and Gehrke, 2003) ที่ได้กล่าวถึงบ่อยๆ ดังต่อไปนี้

Student (*cid:* string, *sid:* string, *name:* string, *login:* string, *age:* integer, *gpa:* real)

Faculty (*fid:* string, *fname:* string, *sal:* real)

Course (*cid:* string, *cname:* string, *credits:* integer)

Room (*rno:* integer, *address:* string, *capacity:* integer)

Enrolled (*sid:* string, *cid:* string, *grade:* float)

Teaches (*fid:* string, *cid:* string)

Meets_In (*cid:* string, *rno*: integer, *time:* string)

กรณีตัวอย่างของข้อมูลแสดงไว้เมื่อมีการยกตัวอย่างการใช้ SQL

8.2.1 ข้อคำถามอย่างง่าย (Simple Queries)—SELECT, WHERE

<u>แสดงข้อมูลทุกระเบียน</u>

ตัวอย่างที่ 8.1 SELECT

แสดงข้อมูลทุกระเบียน ทุกคอลัมน์ คำสั่ง SQL สำหรับแสดงข้อมูลทุกระเบียนและทุกคอลัมน์ จะไม่มีการใช้ WHERE

SELECT cid, sid, name, login, age, gpa

FROM Student;

หรือใช้เครื่องหมาย “*” แทนการแสดงข้อมูลทุกคอลัมน์

SELECT *

FROM Student;

จากตัวอย่างเป็นการแสดงข้อมูลทุกๆ คอลัมน์ โดยการระบุชื่อคอลัมน์ทั้งหมดหรือใช้เครื่องหมาย “*” แทน ซึ่งเป็นการแสดงข้อมูลทุกระเบียนและทุกคอลัมน์จากตารางนักศึกษาได้ผลลัพธ์ดังนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

ตัวอย่างที่ 8.2 SELECT

แสดงข้อมูลทุกระเบียน แต่แสดงเฉพาะบางคอลัมน์ คำสั่ง SQL สำหรับแสดงข้อมูลทุกระเบียน จะไม่มีการใช้ WHERE ในคำสั่ง สำหรับการระบุคอลัมน์นั้นจะทำการระบุหลัง SELECT

SELECT sid, name, gpa

FROM Student

จากตัวอย่างเป็นการแสดงข้อมูลทุกๆ ระเบียน สำหรับคอลัมน์นั้นแสดงเพียงคอลัมน์ รหัสนักศึกษา ชื่อนักศึกษา และเกรดเฉลี่ยเท่านั้น ได้ผลลัพธ์ดังนี้ ซึ่งผลลัพธ์ที่แสดงไม่ได้มีการเรียงลำดับตามคอลัมน์ใดๆ หรือในกรณีทั่วไประบบจัดการฐานข้อมูลจะแสดงข้อมูลเรียงตามลำดับของคีย์หลัก

Student

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

ตัวอย่างที่ 8.3 DISTINCT

การแสดงข้อมูลที่ไม่ซ้ำกันโดยการใช้ DISTINCT

แสดงรหัสนักศึกษาทั้งหมดที่ได้ลงทะเบียนเรียน

SELECT sid

FROM enrolled;

แสดงรหัสนักศึกษาทั้งหมดที่เคยลงทะเบียนเรียน โดยรหัสนักศึกษาที่ซ้ำกันแสดงเพียงครั้งเดียวเท่านั้นโดยการใช้ DISTINCT

SELECT DISTINCT sid

FROM enrolled;

ได้ผลลัพธ์ดังนี้ ตารางซ้ายมือแสดงข้อมูลทั้งหมดในตาราง Enrolled ในขณะที่ตารางกลางแสดงข้อมูลตามคำสั่ง SQL ที่ไม่ได้ใช้ DISTINCT และตารางทางขวาแสดงข้อมูลที่ได้จากการใช้ DISTINCT ทั้งนี้จะพบว่า รหัสนักศึกษา B5075688 จะแสดงเพียง 1 ระเบียนเท่านั้น

Enrolled

cid	grade	sid
204203	4	B5075650
204204	3	B5075831
204205	1	B5075688
204329	0	B5075666
204204	4	B5075688

ข้อมูลทั้งหมดในตาราง Enrolled

Enrolled

sid
B5075650
B5075831
B5075688
B5075666
B5075688

รหัสนักศึกษาทั้งหมดในการลงทะเบียน

Enrolled

sid
B5075650
B5075831
B5075688
B5075666

รหัสนักศึกษาที่เคยลงทะเบียนโดยไม่แสดงรหัสนักศึกษาที่ซ้ำกัน

ตัวอย่างที่ 8.4 การคำนวณ

การแสดงข้อมูลโดยมีการคำนวณ

แสดงรายการรหัสนักศึกษา รายชื่อนักศึกษา และผลการเรียนของนักศึกษาเป็นร้อยละจากเกรดเฉลี่ย

SELECT sid, name, gpa*100/4

FROM Student;

ข้อมูลที่ได้จากคำสั่ง SQL ดังกล่าวปรากฏในคอลัมน์หรือฟิลด์สุดท้ายเรียกว่าฟิลด์ที่ได้จากการคำนวณ (calculated filed or computed field or derived field) ในตัวอย่างเป็นการนำเอาเกรดเฉลี่ยในรูปแบบเต็ม 4.00 มาคำนวณให้เป็นร้อยละ โดยการคูณด้วย 100 และหารด้วย 4 ซึ่งฟิลด์ที่ได้จากการคำนวณนี้จะไม่มีชื่อฟิลด์ ผลลัพธ์มักจะเป็นลำดับที่ของคอลัมน์ ในกรณีนี้คือ col3 ซึ่งเราสามารถกำหนดชื่อของคอลัมน์ที่ได้จากการคำนวณโดยการใช้ AS ดังนี้

SELECT sid, name, gpa*100/4 AS percentGrade

FROM Student;

ซึ่งการคำนวณนั้นสามารถใช้เครื่องหมาย บวก ลบ คูณ หาร วงเล็บ ซึ่งการคำนวณนอกจากจะคำนวณจากคอลัมน์เดียวแล้ว ยังสามารถคำนวณได้จากหลายคอลัมน์ โดยค่าของคอลัมน์จะต้องเป็นตัวเลขเท่านั้น จึงจะสามารถคำนวณได้ คำสั่ง SQL ตามตัวอย่าง สร้างตารางผลลัพธ์ดังนี้

Student

sid	name	percentGrade
B5075666	สมชาย	86.00
B5075688	สมศรี	80.25
B5075650	สมศรี	95.50
B5075831	สมศักดิ์	45.00
B5075832	สมน้ำหน้า	50.00

การแสดงข้อมูลบางระเบียน Row selection (WHERE clause)

เป็นการแสดงข้อมูลจากตารางข้อมูลเฉพาะระเบียนที่ตรงตามเงื่อนไข โดยสามารถกำหนดเงื่อนไขในรูปแบบของนิพจน์ต่างๆ ประกอบการใช้ WHERE ซึ่งรูปแบบของเงื่อนไขนั้นอาจมีลักษณะดังนี้

- การเปรียบเทียบค่าของนิพจน์หนึ่งกับค่าของอีกนิพจน์หนึ่ง
- การทดสอบค่าของนิพจน์หนึ่งแบบเป็นช่วงกับช่วงค่าที่ระบุ
- การทดสอบว่าค่าที่ระบุเป็นสมาชิกหรือมีค่าเท่ากับสมาชิกในเซตที่ระบุไว้
- การตรวจความเหมือนกันของรูปแบบสายอักขระตามรูปแบบที่กำหนด
- การทดสอบว่าคอลัมน์นั้นๆ มีค่าว่าง หรือไม่มีค่าว่าง

การดำเนินการ WHERE นั้นเป็นลักษณะของการดำเนินการ selection ในพีชคณิตเชิงสัมพันธ์

ตัวอย่างที่ 8.5 WHERE clause

เงื่อนไขการแสดงข้อมูลแบบเปรียบเทียบ

แสดงข้อมูลนักศึกษาทั้งหมดที่มีเกรดเฉลี่ยมากกว่า 2.00

```
SELECT cid, sid, name, login, age, gpa
FROM Student
WHERE gpa > 2;
```

คำสั่ง SQL ตัวอย่างแสดงข้อมูลนักศึกษาทุกคอลัมน์จากตารางนักศึกษา ในส่วนของช่วงคำสั่ง WHERE นั้นกำหนดเงื่อนไข gpa > 2 หมายถึงให้แสดงข้อมูลเฉพาะระเบียนที่ค่าในคอลัมน์ gpa มีค่ามากกว่า 2 ได้ผลลัพธ์ดังนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82

เครื่องหมายที่สามารถใช้ในนิพจน์สำหรับเงื่อนไขการแสดงข้อมูลได้แก่

เท่ากับ	=		มากกว่าหรือเท่ากับ	> =
ไม่เท่ากับ	< > หรือ !=		และ	AND
น้อยกว่า	<		หรือ	OR
มากกว่า	>		ไม่	NOT
น้อยกว่าหรือเท่ากับ	< =		วงเล็บเปิดและปิด	( )

การดำเนินการในนิพจน์จะกระทำจากซ้ายไปขวาโดยมีลำดับการดำเนินการ ( ), NOTs, ANDs, และ ORs ตามลำดับ

ตัวอย่างที่ 8.6 WHERE clause

เงื่อนไขการแสดงข้อมูลแบบเปรียบเทียบหลายการเปรียบเทียบ

แสดงข้อมูลของนักศึกษาทั้งหมดที่มีอายุ 18 หรือ 19 ปี

SELECT *

FROM Student

WHERE age = 18 OR age = 19;

จากตัวอย่างเป็นการแสดงข้อมูลทุกคอลัมน์จากตารางนักศึกษา โดยกำหนดเงื่อนไขแบบเปรียบเทียบแสดงเฉพาะข้อมูลของนักศึกษาที่มีอายุ 18 หรือ 19 ปี โดยใช้ OR ในการเชื่อมการเปรียบเทียบทั้ง 2 ผลลัพธ์จึงเป็นระเบียนของนักศึกษาที่มีอายุ 18 หรือ 19 ปี ดังต่อไปนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82

ตัวอย่างที่ 8.7 BETWEEN/NOT BETWEEN

เงื่อนไขในการแสดงข้อมูลแบบเป็นช่วง (BETWEEN/NOT BETWEEN)

แสดงข้อมูลนักศึกษาที่มีเกรดเฉลี่ยระหว่าง 3.25 ถึง 3.49 ซึ่งเป็นการแสดงนักศึกษาที่มีระดับคะแนนเฉลี่ยอยู่ในช่วงเกียรตินิยมอันดับ 2 ซึ่งในตัวอย่างนี้จะใช้ BETWEEN ซึ่งค่าระหว่างในที่นี้รวมค่าขอบเขตล่างและบนของช่วงด้วย

SELECT sid, name, gpa

FROM student

WHERE gpa BETWEEN 3.25 AND 3.49;

เราสามารถแสดงข้อมูลโดยกำหนดช่วงแบบกลับกันคือแสดงข้อมูลนักศึกษาที่ไม่มีเกรดเฉลี่ยอยู่ในช่วงที่กำหนดโดยใช้ **NOT BETWEEN** ได้ สำหรับตัวอย่างด้านบนนั้นตรงกับการใช้ SQL แบบเปรียบเทียบ 2 การเปรียบเทียบดังนี้

SELECT sid, name, gpa

FROM student

WHERE gpa > = 3.25 AND gpa < = 3.49;

อย่างไรก็ตามจะพบว่าการใช้ BETWEEN นั้นดูง่ายกว่า ซึ่งได้ผลลัพธ์ดังนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44

ตัวอย่างที่ 8.8 IN/NOT IN

เงื่อนไขการแสดงข้อมูลโดยตรวจสอบว่าเป็นสมาชิกในเซตที่กำหนดหรือไม่ (IN/NOT IN)
แสดงรหัสนักศึกษา พร้อมทั้งรายวิชา และเกรด สำหรับการลงทะเบียนที่ได้เกรด A และ B

SELECT sid, cid, grade

FROM enrolled

WHERE grade IN (4, 3);

เราสามารถแสดงข้อมูลโดยกำหนดเงื่อนไขแบบกลับกันคือแสดงข้อมูลนักศึกษาที่ไม่มีเกรดเฉลี่ยอยู่ในเซตที่กำหนดโดยใช้ **NOT IN** ได้ สำหรับตัวอย่างด้านบนนั้นตรงกับการใช้ SQL แบบเปรียบเทียบ 2 การเปรียบเทียบดังนี้

SELECT sid, cid, grade

FROM enrolled

WHERE grade = 4 OR grade = 3;

อย่างไรก็ตามจะพบว่าการใช้ IN นั้นดูง่ายกว่า ซึ่งได้ผลลัพธ์ดังนี้

Enrolled

sid	cid	grade
B5075650	204203	4
B5075831	204204	3
B5075688	204204	4

ตัวอย่างที่ 8.9 LIKE/NOT LIKE

เงื่อนไขการแสดงข้อมูลแบบตรวจความเหมือนกันของรูปแบบสายอักขระตามรูปแบบที่กำหนด (LIKE/NOT LIKE)

แสดงรายวิชาทั้งหมดที่มีคำว่า “Development” อยู่ในชื่อวิชาจากตารางรายวิชาต่อไปนี้

Course

cid	cname	credits
202102	Information Technology I	3
202103	Information Technology II	3
204203	Software Design and Development	4
204204	Database Design and Development	4
204205	Multimedia Design and Development	4
204329	Web Technology	3

สำหรับการแสดงข้อมูลโดยการตรวจสอบความเหมือนกันของรูปแบบสายอักขระนั้นเราจะใช้ LIKE ประกอบกับเครื่องหมายต่างๆ ที่ใช้แทนรูปแบบต่างๆ ของสายอักขระ ได้แก่

% หรือเครื่องหมายร้อยละ แทนอักขระใดๆ ก็ได้มีความยาว 0 ตัวอักษรหรือมากกว่า (wildcard)

_ หรือเครื่องหมายเส้นใต้ แทนอักขระใดๆ ก็ได้มีความยาว 1 ตัวอักษรพอดี

นอกจากตัวอักษรพิเศษทั้ง 2 ตัวแล้ว ตัวอักษรอื่นๆ แทนตัวอักษรของมันเอง

พิจารณาตัวอย่างส่วนของ SQL ที่ใช้ในการตรวจสอบรูปแบบสายอักขระต่อไปนี้

cname LIKE ‘I%’

หมายความว่ารายวิชาใดๆ ที่ชื่อวิชาขึ้นต้นด้วยตัวอักษร I จากนั้นจะตามด้วยตัวอักษรใดๆ ยาวกี่ตัวก็ได้ หรือไม่มีตัวอักษรตามหลังตัว I ก็ได้

cname LIKE ‘I_ _ _’

หมายความว่ารายวิชาใดๆ ที่ชื่อวิชาขึ้นต้นด้วยตัวอักษร I จากนั้นตามด้วยตัวอักษรใดๆ มีความยาวต่อมาจากตัว I 3 ตัวอักษรพอดี ไม่มากหรือน้อยไปกว่านั้น ขอให้สังเกตว่าเครื่องหมายขีดเส้นใต้ทั้ง 3 ไม่มีช่องว่างใดๆ ขั้นระหว่างขีด

cname LIKE ‘%y’

หมายความว่ารายวิชาใดๆ ที่ชื่อวิชาลงท้ายด้วยตัวอักษร y ซึ่งก่อนหน้าตัวอักษร y เป็นตัวอักษรใดๆ มีความยาวเท่าใดก็ได้ หรือไม่มีตัวอักษรหน้าตัว y เลยก็ได้ แต่ขอให้สังเกตว่าจะต้องไม่มีตัวอักษรตามหลังจากอักษร y มิเช่นนั้นจะไม่เข้าเงื่อนไข

cname LIKE ‘%Development%’

หมายความว่ารายวิชาใดๆ ที่ชื่อวิชามีคำว่า Development อยู่ในส่วนใดส่วนหนึ่ง

cname NOT LIKE ‘I%’

สังเกตว่าในกรณีนี้เราไม่ได้ใช้ LIKE เท่านั้น แต่มีการเพิ่ม NOT จึงหมายความว่ารายวิชาใดๆ ที่ชื่อวิชาไม่ได้ขึ้นต้นด้วยตัวอักษร I

เนื่องจากเครื่องหมาย % และ _ เป็นเครื่องหมายพิเศษ ในบางกรณีเราอาจต้องการเปรียบเทียบรูปแบบของสายอักขระที่มีเครื่องหมาย % หรือ _ เช่น เปรียบเทียบค่าของคอลัมน์กับค่า '15%' เราใช้ ESCAPE ประกอบเครื่องหมายอื่นในการแทนเครื่องหมายพิเศษ โดยส่วนของ SQL ต่อไปนี้แสดงการเปรียบเทียบค่าในคอลัมน์กับสายอักขระ '15%'

LIKE '15#%' ESCAPE '#'

ตัวอย่างที่กล่าวมาเป็นเพียงส่วนของ SQL ซึ่งเป็นเงื่อนไขในการเปรียบเทียบรูปแบบของสายอักขระ SQL ต่อไปนี้เป็นตัวอย่างของ SQL เต็มรูปแบบในการแสดงข้อมูลรายวิชาทั้งหมดที่มีคำว่า Development อยู่ในส่วนหนึ่งส่วนใดของชื่อวิชา

SELECT cid, cname, credits

FROM Course

WHERE cname LIKE '%Development%'

ได้ผลลัพธ์ดังนี้

Course

cid	cname	credits
204203	Software Design and Development	4
204204	Database Design and Development	4
204205	Multimedia Design and Development	4

หมายเหตุ ในระบบจัดการฐานข้อมูลบางระบบอาจใช้เครื่องหมายอื่นๆ ในการแทนรูปแบบพิเศษของการตรวจสอบรูปแบบสายอักขระ เช่น ระบบจัดการฐานข้อมูล Microsoft Access ใช้เครื่องหมาย "*" และ "?" แทนเครื่องหมาย "%" และ "_" ตามลำดับ

ตัวอย่างที่ 8.10 IS NULL/IS NOT NULL

เงื่อนไขการแสดงข้อมูลที่เป็นค่าว่าง (IS NULL/IS NOT NULL)

พิจารณาตารางข้อมูลการลงทะเบียนของนักศึกษาต่อไปนี้

Enrolled

cid	grade	sid
204203	4	B5075650
204204	3	B5075831
204205	1	B5075688
204329	0	B5075666
204204	4	B5075688
204329	*NULL*	B5075831

ในกรณีนี้เกรดของนักศึกษารหัส B5075831 ที่ลงทะเบียนเรียนรายวิชา 204329 ในระเบียนสุดท้ายนั้นยังไม่ปรากฏว่าได้เกรดใดในการเรียน ค่าจึงเป็นค่าว่าง หรือค่า NULL ซึ่งหมายความว่าไม่มีค่าใดๆ เลยในคอลัมน์หรือยังไม่ทราบค่า ในกรณีนี้การเปรียบเทียบค่าในคอลัมน์นั้นๆ กับค่าอื่นๆ จึงไม่สามารถกระทำได้ เช่น ให้แสดงข้อมูลการลงทะเบียนเรียนของนักศึกษา B5075831 ที่ยังไม่ได้รับเกรด ลองพิจารณาการเปรียบเทียบเพื่อแสดงข้อมูลดังกล่าวจากส่วนของ SQL ต่อไปนี้

sid = 'B5075831' AND grade = ''

หรือ

sid = 'B5075831' AND grade <> 3

ส่วนของ SQL ทั้งสองไม่สามารถแสดงข้อมูลได้ โดยในกรณีที่ 1 '' จะไม่เท่ากับค่าว่างหรือ NULL และในกรณีที่ 2 เป็นการสื่อความหมายที่ไม่ถูกต้อง เพราะต่อไปนักศึกษาคนดังกล่าวอาจได้เกรดจากวิชาอื่นที่ไม่ใช่ 3 จะทำให้การแสดงข้อมูลไม่ถูกต้อง การทดสอบโดยใช้ 0 ยิ่งไม่ถูกต้องเพราะ 0 เป็นเกรดหนึ่งของผลการเรียน การทดสอบค่าว่าง เราใช้ **IS NULL** ดังนี้

SELECT *

FROM Course

WHERE sid = 'B5075831' AND grade IS NULL;

เป็นการแสดงข้อมูลทั้งหมดในการลงทะเบียนแต่ละระเบียนของนักศึกษารหัส B5075831 ที่ยังไม่ทราบเกรด ซึ่งได้ผลลัพธ์ดังนี้

Enrolled

cid	grade	sid
204329	*NULL*	**B5075831**

8.2.2 การเรียงลำดับผลลัพธ์ (ORDER BY)

ผลลัพธ์ที่ได้จากการแสดงข้อมูลด้วยคำสั่ง SQL นั้นปกติจะไม่ได้เรียงตามลำดับใดๆ ตามข้อมูลในคอลัมน์ใดๆ เลย เว้นแต่ในบางระบบจัดการฐานข้อมูลที่ข้อมูลจะเรียงตามคีย์หลักของตารางนั้นๆ เราสามารถกำหนดให้การแสดงผลลัพธ์เรียงตามลำดับของข้อมูลในคอลัมน์ใดๆ ได้ ซึ่งสามารถที่จะเรียงลำดับจากน้อยไปมาก หรือมากไปน้อยก็ได้ โดยใช้ **ASC** และ **DESC** ตามลำดับ นอกจากนี้แล้วเราสามารถกำหนดให้ข้อมูลที่แสดงเรียงลำดับตามคอลัมน์ที่มีอยู่ในระเบียนแต่ไม่ได้ปรากฏอยู่ในผลลัพธ์ก็ได้ การเรียนลำดับนั้นเป็นส่วนหลังสุดของคำสั่ง SQL

ตัวอย่างที่ 8.11 ORDER BY

จากตารางข้อมูลนักศึกษาต่อไปนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

แสดงรหัสนักศึกษาทั้งหมดเรียงตามลำดับเกรดเฉลี่ยจากมากไปน้อย

SELECT sid

FROM Student

ORDER BY gpa DESC;

จากตัวอย่าง ORDER BY gpa เป็นส่วนที่กำหนดการเรียนลำดับของผลลัพธ์ DESC เป็นการกำหนดรูปแบบการเรียนลำดับตาม gpa จากมากไปน้อย มาจากคำว่า descending ซึ่งตรงข้ามกับการใช้รูปแบบ ASC ที่มาจากคำว่า ascending ซึ่งเป็นการเรียนลำดับจากน้อยไปมาก จากตัวอย่างได้ผลลัพธ์ดังนี้

Student

sid	gpa
B5075650	3.82
B5075666	3.44
B5075688	3.21
B5075832	2.00
B5075831	1.80

ผลลัพธ์ที่ได้จากตัวอย่างเป็นตารางข้อมูลที่มีเพียงคอลัมน์เดียว แสดงเฉพาะรหัสนักศึกษา สำหรับคอลัมน์ที่เป็นสีเทานั้นไม่ได้ปรากฏในผลลัพธ์แต่แสดงในที่นี้เพื่อให้เห็นว่าผลลัพธ์ที่แสดงเรียงตามลำดับเกรดเฉลี่ยของนักศึกษาแต่ละคนจากมากไปหาน้อย โดยเกรดเฉลี่ยไม่จำเป็นต้องปรากฏในผลลัพธ์ก็ได้

ตัวอย่างที่ 8.12 MULTI-COLUMN ORDER BY

การเรียงลำดับข้อมูลโดยกำหนดค่าในคอลัมน์ที่ต้องการเรียงลำดับหลายคอลัมน์

แสดงข้อมูลนักศึกษาทั้งหมดเรียงตามชื่อนักศึกษาตามด้วยอายุจากมากไปหาน้อย

```
SELECT *
FROM Student
ORDER BY name, age DESC;
```

ได้ผลลัพธ์ดังนี้

Student

cid	sid	name	login	age	gpa
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00
3100904022132	B5075666	สมชาย	somchai@it	18	3.44

จากผลลัพธ์ ข้อมูลจะเรียงตามชื่อนักศึกษา จากมากไปน้อย ซึ่งสมศักดิ์จะมาเป็นลำดับสุดท้ายตามพจนานุกรมในขณะที่สมชายจะมาก่อน จากข้อมูลที่มีทั้งหมด อย่างไรก็ตามบ่อยครั้งที่มักมีค่าที่สามารถซ้ำกันได้ในคอลัมน์ เราสามารถกำหนดเพิ่มเติมได้อีกว่าในกรณีที่ค่าในคอลัมน์ที่ใช้เรียงลำดับซ้ำกัน ต่อไปจะให้เรียงลำดับตามคอลัมน์ใด ในตัวอย่างนี้เรากำหนดให้ใช้อายุเป็นคอลัมน์ถัดมาที่ใช้ในการเรียงลำดับ ซึ่งสังเกตได้จากสมศรีที่มี 2 คน สมศรีที่มีอายุ 19 ปีมาก่อนสมศรีที่มีอายุ 18 ปี

8.2.3 การดำเนินการเป็นชุด (SQL Aggregate Functions)—COUNT, SUM, AVG, MIN/MAX

การดำเนินการเป็นชุดในคำสั่ง SQL อาศัยฟังก์ชันต่างๆ ในลักษณะเดียวกันกับการดำเนินการเป็นชุดในพีชคณิตเชิงสัมพันธ์ ซึ่ง International Organization for Standardization (ISO) ได้กำหนดฟังก์ชันมาตรฐานไว้ดังนี้

COUNT—นับจำนวนของค่าในคอลัมน์	AVG—หาค่าเฉลี่ยของค่าในคอลัมน์
SUM—หาผลรวมของค่าในคอลัมน์	MIN/MAX—หาค่าต่ำสุด/สูงสุดของคอลัมน์

โดยมีข้อกำหนดเพิ่มเติมดังนี้

- ฟังก์ชันการดำเนินการแบบชุดทั้งหมดสามารถดำเนินการได้กับคอลัมน์ 1 คอลัมน์เท่านั้น
- สำหรับฟังก์ชัน **COUNT, MIN, MAX** ไม่จำเป็นต้องดำเนินการกับค่าที่เป็นตัวเลขก็ได้
- การใช้ **COUNT(*)** นั้น เป็นการนับระเบียนทุกระเบียน ซึ่งรวมระเบียนที่มีค่า NULL ซึ่งโดยปกติการใช้ COUNT กับคอลัมน์ใดๆ จะไม่นับระเบียนที่คอลัมน์ที่กำหนดมีค่าเป็น NULL ซึ่ง COUNT(*) นั้นโดยแท้จริงแล้วคือการนับระเบียนของคอลัมน์ใดๆ แต่ละระเบียนจะต้องมีอย่างน้อย 1 คอลัมน์ที่ไม่มีค่าเป็น NULL จึงทำให้ COUNT(*) นับระเบียนที่มีค่า NULL ด้วย

- เราสามารถใช้ **DISTINCT** ประกอบฟังก์ชัน โดยวาง DISTINCT ไว้ก่อนหน้าชื่อคอลัมน์ที่จะดำเนินการเพื่อขจัดระเบียนที่มีค่าซ้ำซ้อนออกก่อน
- เราสามารถใช้ **ALL** ประกอบฟังก์ชัน เพื่อให้ฟังก์ชันดำเนินการกับระเบียนทุกระเบียนแม้จะมีค่าซ้ำกัน ซึ่งปกติฟังก์ชันดำเนินการเสมือนมี ALL อยู่แล้ว
- DISTINCT ไม่มีผลกับฟังก์ชัน MIN และ MAX แต่ผลมีกับ SUM และ AVG
- DISTINCT ใช้ได้เพียง 1 ครั้งในฟังก์ชันดำเนินการเป็นชุด
- ฟังก์ชันการดำเนินการเป็นชุดใช้ได้กับคำสั่ง **SELECT** และส่วนของคำสั่ง **HAVING** เท่านั้น

ซึ่งสามารถยกตัวอย่างการใช้งานฟังก์ชันต่างๆ ดังด้านล่างด้วยข้อมูลของนักศึกษาดังตารางต่อไปนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

Enrolled

cid	grade	sid
204203	4	**B5075650**
204204	3	**B5075831**
204205	1	**B5075688**
204329	0	**B5075666**
204204	4	**B5075688**
204329	*NULL*	**B5075831**

Room

rno	address	capacity
1101	Lecture Building	30
1102	Lecture Building	30
1201	Lecture Building	120
2101	Lecture Building	300
3101	Lecture Building	300
4101	Lecture Building	1500

ตัวอย่างที่ 8.13 COUNT

การนับระเบียนทั้งหมด **COUNT(*)**

SELECT COUNT(*) AS myCount FROM Student;

เป็นการนับจำนวนนักศึกษาทั้งหมดในตารางได้ผลลัพธ์ดังนี้

Student

myCount
5

ตัวอย่างที่ 8.14 COUNT(DISTINCT)

การนับระเบียนที่มีค่าไม่ซ้ำกัน **COUNT(DISTINCT)**

SELECT COUNT(cid) AS enrolledCount, COUNT(DISTINCT cid) AS courseCount

FROM enrolled;

COUNT(cid) เป็นการนับจำนวนรายวิชาทั้งหมดที่ทราบค่า และ COUNT(DISTINCT cid) เป็นการนับจำนวนรายวิชาทั้งหมดที่มีเคยมีนักศึกษาลงทะเบียน โดยตัดรหัสวิชาที่ซ้ำกันให้เหลือเพียง 1 ระเบียนในการนับ จากตารางการลงทะเบียน ได้ผลลัพธ์ดังนี้

Student

enrolledCount	courseCount
6	4

ผลลัพธ์ที่ได้เป็นตาราง 2 คอลัมน์ ซึ่ง enrolledCount เกิดจาก COUNT(cid) เป็นการนับรายวิชาทั้งหมดแม้จะซ้ำกันส่วน courseCount เกิดจาก COUNT(DISTINCT cid) ได้ค่าเพียง 4 เนื่องจากรายวิชา 204204 และ 204329 ปรากฏอยู่ในตารางการลงทะเบียนรายวิชาละ 2 ระเบียน เมื่อตัดระเบียนที่ซ้ำกันออกด้วย DISTINCT จึงเหลือเพียง 4 ระเบียน

ตัวอย่างที่ 8.15 COUNT และ SUM

การนับและการหาผลรวม **COUNT** และ **SUM**

SELECT COUNT(*) AS roomCount, SUM(capacity) AS seatCount

FROM Room

จากตัวอย่าง COUNT(*) เป็นการนับจำนวนระเบียนที่มีอยู่ในตารางห้องซึ่งผลลัพธ์จะปรากฏอยู่ในคอลัมน์ roomCount ซึ่งเป็นการนับจำนวนห้องนั่นเอง ส่วน SUM(capacity) เป็นการหาผลรวมของความจุทุกๆ ห้อง ได้เป็นที่นั่งรวมทั้งหมด ดังผลลัพธ์ต่อไปนี้

Student

roomCount	seatCount
6	2280

ตัวอย่างที่ 8.16 MIN, MAX และ AVG

การหาค่าน้อยที่สุด มากที่สุด และค่าเฉลี่ย **MIN, MAX** และ **AVG**

SELECT MIN(gpa) AS minGpa, MAX(gpa) AS maxGpa, AVG(gpa) AS avgGpa

FROM Student;

ได้ผลลัพธ์ดังนี้

Student

minGpa	maxGpa	avgGpa
1.80	3.82	2.854

8.2.4 การจัดกลุ่ม (Grouping)—GROUP BY

ข้อคำถามแบบกลุ่มนั้นเราจะใช้ **GROUP BY** ในการจัดกลุ่ม โดยระบุกลุ่มของการดำเนินการตามคอลัมน์ ซึ่งค่าที่ได้จากการดำเนินการจะเป็นค่าเดียวสำหรับแต่ละกลุ่ม

ตัวอย่างที่ 8.17 GROUP BY

การใช้ GROUP BY แสดงรหัสวิชาและจำนวนผู้ลงทะเบียนในแต่ละรายวิชา

SELECT cid, COUNT(cid) AS studentCount

FROM enrolled

GROUP BY cid;

จากตัวอย่างมีการใช้ฟังก์ชันการดำเนินการแบบกลุ่มได้แก่ COUNT ซึ่งเป็นการนับจำนวนรหัสวิชาจากตารางลงทะเบียน ในกรณีนี้มีการใช้ GROUP BY cid เป็นการนับจำนวนรายวิชาแล้วแสดงผลลัพธ์แยกเป็นผลรวมของจำนวนระเบียนของแต่ละรายวิชา ได้ผลลัพธ์ดังนี้

Enrolled

cid	studentCount
204203	1
204204	2
204205	1
204329	2

<u>การบังคับการจัดกลุ่ม (Restricting groupings—HAVING clause)</u>

ในกรณีที่มีการแสดงข้อมูลโดยการจับกลุ่มของข้อมูล เราสามารถกรองกลุ่มของข้อมูลโดยกำหนดเงื่อนไขที่แต่ละกลุ่มจะต้องคุณสมบัติตามที่กำหนดได้ โดยใช้ **HAVING** ซึ่งมีลักษณะคล้ายคลึงกันกับการใช้ **WHERE** เพื่อกรองแถวของข้อมูล แต่ **HAVING** นี้จะใช้กับ **GROUP BY** ในการกรองกลุ่มของข้อมูล

ในส่วนของประโยค HAVING นี้จะต้องมีการใช้คำสั่งการดำเนินการแบบเป็นชุด (aggregate function) อย่างน้อย 1 คำสั่ง มิเช่นนั้นจะไม่ได้เป็นการกรองข้อมูลแบบเป็นกลุ่ม หากเป็นเช่นนั้นเงื่อนไขการค้นหาและกรองข้อมูลใน HAVING ต้องย้ายไปอยู่ในส่วนของประโยค WHERE เพื่อกรองระเบียนข้อมูล และให้ระลึกว่าเราไม่ใช้การดำเนินการแบบเป็นชุดในส่วนของประโยค WHERE

ตัวอย่างที่ 8.18 HAVING

การใช้ **HAVING**

SELECT age, MAX(gpa) AS maxGpa

FROM Student

GROUP BY age HAVING MAX(gpa) >= 3.25;

จากตัวอย่างเป็นการแสดงข้อมูลนักศึกษาโดยแสดงอายุกับคะแนนสูงสุดของนักศึกษาจำแนกตามอายุนักศึกษา โดยการจำแนกตามอายุนั้นได้แก่การใช้ GROUP BY age นอกจากนี้ยังระบุด้วยว่าแสดงเฉพาะกลุ่มอายุที่มีคะแนนสูงสุดมากกว่าหรือเท่ากับ 3.25 ซึ่งได้แก่การใช้ HAVING MAX(gpa) >= 3.25 ได้ผลลัพธ์ดังนี้

Student

age	gpa
18	3.44
19	3.82

8.2.5 ข้อคำถามรอง (Subqueries)—HAVING

ข้อคำถามรองหรือข้อคำถามย่อยเป็นการใช้คำสั่ง SELECT ซ้อนคำสั่ง SELECT อีกทีหนึ่งเพื่อให้คำสั่ง SELECT หลักสมบูรณ์ โดยเราเรียกคำสั่ง SELECT ที่เป็นคำสั่งรองอยู่ภายใต้คำสั่ง SELECT อื่นอีกทีหนึ่งว่า **Inner SELECT** หรือ **SubSELECT** ซึ่งจะนำไปทำให้คำสั่งหลักหรือคำสั่ง SELECT ที่อยู่ภายนอกสมบูรณ์ซึ่งเราเรียกคำสั่ง SELECT หลักนี้ว่า **Outer SELECT**

สำหรับการใช้งานคำสั่ง SELECT ย่อยนี้เราจะใช้ประกอบ WHERE และ HAVING ใน Outer SELECT ซึ่งเราจะเรียกประโยค WHERE และ HAVING ที่บรรจุ Inner SELECT นี้ว่าข้อคำถามรอง (subquery หรือ nested query) ซึ่งสามารถใช้ได้ในคำสั่ง **INSERT, UPDATE** และ **DELETE** ได้เช่นเดียวกัน

ซึ่งข้อคำถามย่อยนี้มีหลายประเภทจำแนกตามรูปแบบของผลลัพธ์ของข้อคำถามย่อยนั้น ดังนี้

1) **Scalar subquery** ผลลัพธ์ที่ได้เป็นค่าสเกลาร์ เป็นค่าเดี่ยว อยู่ในตาราง 1 แถวและ 1 คอลัมน์เท่านั้น
2) **Row subquery** ผลลัพธ์ที่ได้เป็นลักษณะของแถว 1 แถว มีหลายคอลัมน์
3) **Table subquery** ผลลัพธ์ที่ได้เป็นตาราง 1 หรือหลายคอลัมน์ หลายแถว เช่นการนำไปใช้ประโยชน์ในการใช้ **IN**

ตัวอย่างที่ 8.19 Subqueries

ข้อคำถามย่อยโดยเปรียบเทียบความเท่ากัน

```
SELECT sid, grade
FROM  enrolled
WHERE cid = (SELECT cid
        FROM Course
        WHERE cname = 'Database Design and Development'
```

จากตัวอย่างเป็นการแสดงข้อมูลรหัสนักศึกษาและเกรดที่ได้ของรายวิชาที่มีค่าเท่ากับข้อคำถามย่อย ซึ่งข้อคำถามย่อยในที่นี้คือการค้นหารหัสวิชาที่มีชื่อวิชาว่า 'Database Design and Development' ตัวอย่างข้างต้นจึงมีค่าเท่ากับการใช้คำสั่ง

```
SELECT sid, grade
FROM enrolled
WHERE cid = '204204'
```

ได้ผลลัพธ์ดังนี้

Enrolled

sid	grade
B5075831	3
B5075688	4

เราสามารถมองผลลัพธ์ที่ได้จากการใช้ข้อคำถามย่อยว่าเป็นตารางชั่วคราวก็ได้ ซึ่งนอกจากการเปรียบความความเท่ากันโดยใช้เครื่องหมาย "=" แล้ว เรายังสามารถใช้เครื่องหมายเปรียบเทียบอื่นๆ ได้แก่ <, >, <=, >=, <> ได้ ขอให้ระลึกไว้เสมอว่าข้อคำถามย่อยใช้ในประโยค WHERE หรือ HAVING เท่านั้น โดยอาศัยเครื่องหมายวงเล็บ "(…)" เพื่อครอบข้อคำถามย่อยไว้

ตัวอย่างที่ 8.20 Subqueries

การใช้ข้อคำถามย่อยกับฟังก์ชันการดำเนินการแบบเป็นชุด

SELECT sid, name, gpa - (SELECT AVG(gpa) FROM Student) AS gpaDiff

FROM Student

WHERE gpa > (SELECT AVG(gpa) FROM Student);

จากตัวอย่างเป็นการแสดงข้อมูลรหัสนักศึกษา ชื่อนักศึกษา และ gpa – (SELECT AVG(gpa) FROM Student) คือการแสดงข้อมูลผลต่างระหว่าง gpa ของนักศึกษาแต่ละคนกับ gpa เฉลี่ยของนักศึกษาทั้งหมด ทั้งนี้ในประโยค WHERE เป็นการระบุว่าให้แสดงเฉพาะข้อมูลของนักศึกษาที่มี gpa มากกว่า gpa เฉลี่ยของนักศึกษาทั้งหมด ซึ่งมีค่าเท่ากับการใช้คำสั่ง SQL ต่อไปนี้

SELECT sid, name, gpa – 2.854 AS gpaDiff

FROM Student

WHERE gpa > 2.854;

ได้ผลลัพธ์ดังต่อไปนี้

Student

sid	name	gpaDiff
B5075666	สมชาย	0.586
B5075688	สมศรี	0.356
B5075650	สมศรี	0.966

เกณฑ์ในการใช้ข้อคำถามย่อยสามารถสรุปเป็นกฎ 4 ข้อได้ดังนี้

1) ORDER BY จะไม่ใช้ในข้อคำถามย่อย แต่จะใช้ได้กับ outer SELECT ที่อยู่ชั้นนอกสุดเท่านั้น
2) การใช้ข้อคำถามย่อยเพื่อให้ outer SELECT เปรียบเทียบรายการของข้อมูลที่จาก inner SELECT นั้น inner SELECT จะต้องมีการแสดงข้อมูลอย่างน้อย 1 นิพจน์ (คอลัมน์) ยกเว้นการตรวจสอบโดยการใช้ EXISTS
3) การอ้างถึงคอลัมน์ใดๆ ในข้อคำถามย่อยจะเป็นการอ้างถึงคอลัมน์จากตารางใน FROM ของข้อคำถามย่อยนั้นเอง กรณีที่ต้องการอ้างถึงคอลัมน์จากตารางของ Outer FROM ต้องมีการกำหนดชื่อคอลัมน์ (qualifying column name) ใน Outer FROM เพื่อนำมาอ้างถึงในข้อคำถามย่อย
4) ข้อคำถามย่อยจะต้องอยู่ทางด้านขวาของตัวดำเนินเปรียบเทียบเสมอ ยกตัวอย่างเช่น

   SELECT sid, name, gpa - (SELECT AVG(gpa) FROM Student) AS gpaDiff

   FROM Student

   WHERE (SELECT AVG(gpa) FROM Student) < gpa;

   เป็นการสร้างคำสั่ง SQL ที่ไม่ถูกต้องเนื่องจากข้อคำถามย่อยใน WHERE นั้นอยู่ทางด้านซ้ายของเครื่องหมายเปรียบเทียบ “<”

ตัวอย่างที่ 8.21 Nested IN

การใช้ข้อคำถามย่อยกับ IN **(Nested IN)**

```
SELECT sid, name, gpa
FROM Student
WHERE sid IN(SELECT sid
        FROM enrolled
        WHERE cid = '204204')
```

จากตัวอย่างเป็นการแสดงข้อมูลรหัสนักศึกษา ชื่อนักศึกษา และเกรดเฉลี่ยของนักศึกษาที่ลงทะเบียนเรียนรายวิชา 204204 ซึ่งในส่วนของ WHERE sid IN หมายถึงการแสดงข้อมูลของนักศึกษาที่มีรหัสนักศึกษาปรากฏอยู่ในรายการของการ SELECT ในข้อคำถามย่อย สำหรับข้อคำถามย่อย (SELECT sid FROM Student WHERE cid = '204204') เป็นการแสดงรายการรหัสนักศึกษาทั้งหมดที่ลงทะเบียนรายวิชา 204204 ได้ผลลัพธ์ดังนี้

Student

sid	name	gpa
B5075688	สมศรี	3.21
B5075831	สมศักดิ์	1.80

คำสั่งที่ได้ยกตัวอย่างสามารถเขียนให้อยู่ในรูปของการ JOIN กันของตารางได้ดังนี้

```
SELECT Student.sid, name, gpa
FROM Student, enrolled
WHERE Student.sid = enrolled.sid
AND cid = '204204'
```

ซึ่งจะได้อธิบายในหัวข้อที่ 8.2.7 ข้อคำถามหลายตาราง

8.2.6 มีบ้าง และ ทั้งหมด (ANY/SOME and ALL)

ANY และ ALL เป็นการใช้งานข้อคำถามย่อยเพื่อกำหนดเงื่อนไขในการแสดงข้อมูล โดยคำสั่ง SQL ที่มีการใช้ ANY หรือ ALL จะแสดงข้อมูลเฉพาะแถวที่มีค่าของคอลัมน์เป็นไปตามเงื่อนไขที่กำหนดในทำนองเดียวกับการใช้เครื่องหมายเปรียบเทียบค่า แต่ ANY และ ALL นำมาใช้ประกอบการทดสอบค่าของคอลัมน์หนึ่ง กับชุดของค่าที่นำมาเปรียบเทียบกับค่าทางด้านซ้ายของเครื่องหมายเปรียบเทียบ โดยค่าของนิพจน์จะเป็นจริงสำหรับ ANY ก็ต่อเมื่อ การผลทดสอบโดยเครื่องหมายเปรียบเทียบระหว่างค่าทางซ้าย กับแต่ละค่าในชุดข้อมูลทางด้านขวาของเครื่องหมายเปรียบเทียบเป็นจริงอย่างน้อยหนึ่งค่า ANY สามารถใช้คำว่า SOME แทนได้ สำหรับ ALL ค่าของนิพจน์จะเป็นจริงก็ต่อเมื่อการเปรียบเทียบทุกๆ ค่าระหว่างค่าทางซ้ายของเครื่องหมายเปรียบเทียบกับทุกค่าในชุดของข้อมูลทางขวาของเครื่องหมายเปรียบเทียบเป็นจริงทั้งหมดทุกกรณี

ANY และ ALL ใช้กับข้อคำถามย่อยแสดงข้อมูลผลลัพธ์เป็นข้อมูล 1 คอลัมน์เท่านั้น เนื่องจากคำสั่ง SQL ที่ใช้งาน ANY และ ALL มีความซับซ้อน จึงจำเป็นต้องอาศัยตัวอย่างเพื่อแสดงการใช้งานดังนี้

ตัวอย่างที่ 8.22 SOME

แสดงรหัสนักศึกษา ชื่อนักศึกษา และเกรดเฉลี่ย สำหรับนักศึกษาที่มีเกรดเฉลี่ยสูงกว่าหรือเท่ากับเกรดของนักศึกษาอย่างน้อย 1 คนที่ลงทะเบียนเรียนรายวิชา 204204

```
SELECT sid, name, gpa
FROM Student
WHERE gpa >= SOME (SELECT grade
        FROM enrolled
        WHERE cid = '204204');
```

จากตัวอย่าง คำสั่ง SQL ในส่วน **SELECT sid, name, gpa FROM Student WHERE …** เป็นข้อคำถามหลักแสดงข้อมูลรหัสนักศึกษา ชื่อนักศึกษา และเกรดเฉลี่ยจากตารางนักศึกษา ซึ่งเป็นไปตามเงื่อนไขของ **WHERE gpa > SOME (…)** สำหรับเงื่อนไขดังกล่าวเป็นการทดสอบว่าเกรดเฉลี่ยของนักศึกษานั้นมากกว่าหรือเท่ากับค่าใดๆ ในชุดข้อมูลด้านขวาของเครื่องหมาย “>=” อย่างน้อย 1 ค่าหรือไม่ ซึ่งค่าทางด้านขวาของเครื่องหมายมากกว่าหรือเท่ากับนั้นคือเกรดของรายวิชา 204204 (ขอให้สังเกตว่าเกรดของรายวิชา 204204 นั้นเป็นเกรดที่ได้จากการลงทะเบียนเรียนรายวิชานั้นโดยเฉพาะ ไม่ใช่เกรดเฉลี่ย) ได้มาจากข้อคำถามย่อย **SELECT grade FROM enrolled WHERE cid = '204204'** ลองพิจารณาดูกรณีตัวอย่างข้อมูลดังแสดงด้านล่าง ผลลัพธ์ของข้อคำถามย่อยนี้เป็นการแสดงเกรดหรือระดับคะแนนตัวเลขทั้งหมดของผู้ลงทะเบียนเรียนรายวิชา 204204 ซึ่งมี 2 เกรดได้แก่ 3 และ 4 ซึ่งเป็นเกรดของนักศึกษารหัส B5075831 และ B5075688 ตามลำดับ

จากนั้นเรานำค่าระดับคะแนน {3, 4} (ซึ่งเป็นข้อมูล 1 คอลัมน์เท่านั้น เป็นไปตามรูปแบบของการใช้ ANY/SOME) มาเปรียบเทียบกับเกรดเฉลี่ยของนักศึกษาแต่ละคน เสมือนการตรวจสอบด้วยคำสั่ง SQL ดังนี้

WHERE gpa > 3 OR WHERE gpa > 4

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

Enrolled

cid	grade	sid
204203	4	B5075650
204204	**3**	B5075831
204205	1	B5075688
204329	0	B5075666
204204	**4**	B5075688
204329	*NULL*	B5075831

ผลลัพธ์

grade
3
4

ดังนั้นเมื่อเกรดเฉลี่ยของนักศึกษาคนใด มากกว่าหรือเท่ากับระดับคะแนนของนักศึกษาในรายวิชา 204204 อย่างน้อยหนึ่งคน ข้อมูลของนักศึกษาคนนั้นๆ จะแสดงในผลลัพธ์ จากกรณีตัวอย่างข้อมูล สมชาย 1 คน และสมศรีอีก 2 คนรวมเป็น 3 คนนี้ต่างก็มีเกรดเฉลี่ยสูงกว่าระดับคะแนนที่นักศึกษาคนใดๆ ได้รับจากรายวิชา 204204 นั่นคืออย่างน้อยก็มากกว่านักศึกษาที่ได้เกรด 3 เนื่องจากทั้ง 3 คนมีเกรดเฉลี่ยสูงกว่า 3 ทั้งสิ้น ถึงแม้จะไม่ได้มากกว่าหรือเท่าเกรด 4 เงื่อนไขที่ใช้ SOME นี้ก็ยังเป็นจริง ในขณะที่สมศักดิ์และสมน้ำหน้าไม่ได้มีเกรดเฉลี่ยสูงกว่าหรือเท่ากับเกรดใดๆ ของรายวิชา 204204 เลย จึงได้ผลลัพธ์ดังนี้

Student

sid	name	gpa
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82

จากตัวอย่างเราอาจนำไปใช้ประโยชน์เช่นรายวิชา 204204 เป็นรายวิชาที่นักศึกษามักจะได้เกรดสูง หากนักศึกษาคนใดมีเกรดเฉลี่ยสูงกว่านักศึกษาคนใดๆ ที่ได้เกรดจากรายวิชานี้จะปรากฏรายชื่อในผลลัพธ์ อาจนำรายชื่อไปใช้ในการคัดเลือกนักศึกษาเพื่อส่งนักศึกษาเข้าแข่งขันการตอบปัญหาทางวิชาการ เป็นต้น

ตัวอย่างที่ 8.23 ALL

แสดงรหัสนักศึกษา ชื่อนักศึกษา และเกรดเฉลี่ย สำหรับนักศึกษาที่มีเกรดเฉลี่ยสูงกว่าหรือเท่ากับเกรดของนักศึกษาทุกคนที่ลงทะเบียนเรียนรายวิชา 204204

```
SELECT sid, name, gpa
FROM Student
WHERE gpa >= ALL (SELECT grade
        FROM enrolled
        WHERE cid = '204204');
```

ตัวอย่างนี้สามารถอธิบายได้ในทำนองเดียวกันกับตัวอย่างการใช้ ANY/SOME แต่แตกต่างกันตรงที่เงื่อนไขของข้อคำถามย่อย **WHERE gpa >= ALL (…)** จะเป็นจริงก็ต่อเมื่อค่าทางซ้ายมือของเครื่องหมาย ">=" จะต้องมากกว่าหรือเท่ากับค่าทุกๆ ค่าในชุดข้อมูลทางด้านขวาของเครื่องหมาย ">=" ซึ่งจะต้องมากกว่า 3 และ 4 ซึ่งการใช้ข้อคำถามย่อยดังกล่าวเสมือนแสดงข้อมูลจากคำสั่ง SQL ดังนี้

WHERE gpa >= 3 AND gpa >= 4

หากลองพิจารณาดูเกรดเฉลี่ยของนักศึกษาทั้งหมดจะพบว่ามีบางส่วนที่มีเกรดเฉลี่ยมากกว่า 3 แต่มีมีใครเลยที่มีเกรดเฉลี่ยสูงกว่าหรือเท่ากับ 4 ดังนั้นจึงไม่ปรากฏผลลัพธ์เป็นระเบียนข้อมูลของนักศึกษาคนใดเลย เราอาจใช้ประโยชน์จากคำสั่ง SQL ดังกล่าวเพื่อคัดเลือกนักศึกษาที่มีเกรดเฉลี่ยสูงมากๆ เพื่อนำมาติวรายวิชาดังกล่าว เป็นต้น

8.2.7 ข้อคำถามหลายตาราง (Multi-Table Queries)—JOIN

โดยปกติแล้วข้อมูลที่นำมาใช้ประโยชน์จากระบบจัดการฐานข้อมูลเชิงสัมพันธ์มักจะเป็นข้อมูลที่เกิดจากตารางข้อมูลหลายๆ ตาราง เช่น นักศึกษาคนใด ได้เกรดเท่าไร สำหรับรายวิชาที่ระบุ โดยแสดงรายละเอียดชื่อนักศึกษาและชื่อวิชา ซึ่งจะต้องอาศัยข้อมูลจากตารางนักศึกษา ตารางการลงทะเบียน และตารางรายวิชาประกอบกัน การแสดงข้อมูลจากตารางหลายๆ ตารางนี้เป็นการดำเนินการโดยปกติของระบบจัดการฐานข้อมูลเชิงสัมพันธ์ เนื่องจากข้อมูลแต่ละเอนทิตีมีความสัมพันธ์กัน ตลอดจนข้อมูลที่ซ้ำซ้อนกัน เราได้แยกออกเป็นตารางๆ ซึ่งเป็นการดำเนินการที่ได้จากการทำให้เป็นรูปแบบบรรทัดฐาน

การแสดงข้อมูลจากตารางหลายๆ ตารางนั้นสามารถทำได้โดยอาศัยการดำเนินการ JOIN ตารางเช่นเดียวกันกับการดำเนินการ JOIN ในพีชคณิตเชิงสัมพันธ์

Student (*sid:* string, *name:* string, *login:* string, *age:* integer, *gpa:* real)
Faculty (*fid:* string, *fname:* string, *sal:* real)
Course (*cid:* string, *cname:* string, *credits:* integer)
Room (*rno:* integer, *address:* string, *capacity:* integer)
Enrolled (*sid:* string, *cid:* string, *grade:* float)
Teaches (*fid:* string, *cid:* string)
Meets_In (*cid:* string, *rno*: integer, *time:* string)

ตัวอย่างที่ 8.24 การเชื่อมตารางอย่างง่าย (Simple JOIN)

แสดงรหัสอาจารย์ ชื่ออาจารย์ และรหัสรายวิชาที่อาจารย์แต่ละคนสอน

```
SELECT faculty.fid, fname,cid
FROM faculty, teaches
WHERE faculty.fid = teaches.fid;
```

จากตัวอย่างเป็นการแสดงข้อมูล 3 คอลัมน์จากราตารางอาจารย์และการสอน สำหรับ **WHERE faculty.fid = teaches.fid** นั้นเป็นการระบุคอลัมน์ที่เชื่อมตารางทั้งสองเข้าด้วยกันนั่นเอง ซึ่งระบบจัดการฐานข้อมูลจะนำแถวใดๆ ในตารางหลังมาเชื่อมต่อกับแถวในตารางแรกในกรณีที่มีค่าในคอลัมน์ที่ระบุเท่ากัน ในกรณีนี้คือคอลัมน์ fid แสดงการเชื่อมตารางกัน และได้ผลลัพธ์ดังนี้

Faculty

fid	fname	sal
001	แพนเค้ก	40,000
007	สถิตย์	50,000
069	พอลล่า	30,000

Teaches

fid	cid
001	204205
007	204203
007	204204
007	204329
069	202102
069	202103

ผลลัพธ์

fid	fname	cid
001	แพนเค้ก	204205
007	สถิตย์	204203
007	สถิตย์	204204
007	สถิตย์	204329
069	พอลล่า	202102
069	พอลล่า	202103

สำหรับคำสั่ง SQL ที่แสดงข้างต้น เมื่อเราทำการเชื่อมตาราง คอลัมน์ fid ที่ใช้เป็นตัวเชื่อมตารางจะมีค่าซ้ำกัน คือค่า fid จากตาราง faculty และ fid จากตาราง teaches ซึ่งทำให้เกิดการกำกวมขึ้น เราจึงมีความจำเป็นต้องระบุว่าจะแสดง fid จากตารางใด ดังคำสั่งตัวอย่าง **SELECT faculty.fid, …** ในบางกรณีตารางที่นำมาเชื่อมกันอาจมีชื่อของคอลัมน์ที่ซ้ำกัน แม้จะไม่ได้เป็นคอลัมน์ที่ใช้ในการเชื่อมตารางก็ตาม การระบุตารางเจ้าของคอลัมน์ก็มีความจำเป็นเช่นเดียวกัน ซึ่งเราสามารถเขียนคำสั่ง SQL ตามตัวอย่างให้กระชับขึ้นอีกได้ดังนี้

```
SELECT f.fid, fname,cid
FROM faculty f, teaches t
WHERE f.fid = t.fid;
```

โดยการแทนตาราง faculty ด้วยการใช้ตัวแปรอื่นๆ ในการแทนตาราง ในกรณีนี้คือการแทน faculty ด้วย f และ teaches ด้วย t จากนั้นเราสามารถแทน faculty และ teaches ที่ปรากฏอยู่ในคำสั่ง SQL ด้วย f และ t ได้ตามลำดับ นอกจากนี้เราสามารถใช้ JOIN ในการดำเนินการเชื่อมตารางได้แทนคำสั่ง SQL ข้างต้น ซึ่งให้ผลลัพธ์เช่นเดียวกันดังนี้

```
SELECT f.fid, fname,cid
FROM faculty f JOIN teaches t ON f.fid = t.fid;
```

และเนื่องจากคอลัมน์ที่เชื่อมกันนั้นเป็นคอลัมน์ที่มีชื่อเหมือนกัน เราสามารถเขียน SQL ได้ในอีกรูปแบบหนึ่งดังนี้

SELECT f.fid, fname,cid

FROM faculty f JOIN teaches t USING fid;

ในกรณีที่การเชื่อมตารางใช้คอลัมน์ที่เป็นชื่อเดียวกันและเปรียบเทียบค่าเท่ากัน เป็นการดำเนินการในลักษณะของ NATURAL JOIN ในพีชคณิตเชิงสัมพันธ์นั่นเอง ซึ่งสามารถเขียนในรูปของคำสั่ง SQL ดังนี้

SELECT faculty.fid, fname,cid

FROM faculty NATURAL JOIN teaches;

ตัวอย่างที่ 8.25 การเชื่อมตารางและเรียงลำดับข้อมูลในผลลัพธ์

แสดงรหัสอาจารย์ ชื่ออาจารย์ และรหัสรายวิชาที่อาจารย์แต่ละคนสอน โดยเรียงลำดับตามรหัสวิชา

SELECT faculty.fid, fname,cid

FROM faculty, teaches

WHERE faculty.fid = teaches.fid

ORDER BY t.cid;

เราใช้ **ORDER BY** ในการเรียงลำดับซึ่ง ORDER BY จะต้องวางไว้หลังจาก WHERE เช่นเดียวกันกับการใช้ ORDER BY โดยทั่วไป โดยหากมีการกำกวมในชื่อคอลัมน์ที่ใช้ในการเรียงลำดับ ต้องระบุเจ้าของตารางของคอลัมน์นั้นๆ ด้วย

ผลลัพธ์

fid	fname	cid
069	พอลล่า	**202102**
069	พอลล่า	**202103**
007	สถิตย์	**204203**
007	สถิตย์	**204204**
001	แพนเค้ก	**204205**
007	สถิตย์	**204329**

ตัวอย่างที่ 8.26 การเชื่อมตาราง 3 ตาราง (Three-table JOIN)

แสดงรหัสอาจารย์ ชื่ออาจารย์ รหัสวิชา และชื่อวิชาที่อาจารย์แต่ละคนสอน

```
SELECT f.fid, fname, c.cid, cname
FROM faculty f, teaches t, Course c
WHERE f.fid = t.tid AND t.cid = c.cid
ORDER BY f.fid, c.cid;
```

จากตัวอย่างคำสั่ง SQL แสดงข้อมูลจากตาราง 3 ตาราง โดยเชื่อมตาราง Faculty, Teaches และ Course เข้าด้วยกัน ลองพิจารณาข้อมูลแถวแรกของตาราง Faculty อาจารย์แพนเค้กมีรหัสอาจารย์เป็น 001 ซึ่งเมื่อพิจารณาในตาราง Teaches เราพบว่ารหัสอาจารย์ 001 สอนรายวิชา 204205 และเมื่อนำค่ารหัสวิชา 204205 มาเปรียบเทียบในตาราง Course พบว่าเป็นรายวิชา Multimedia Design and Development จึงได้ข้อมูลครบทั้งหมดที่ต้องการแสดงสำหรับอาจารย์คนแรก ซึ่งการเชื่อมตารางและผลลัพธ์ทั้งหมดแสดงได้ดังต่อไปนี้

Faculty

fid	fname	sal
001	แพนเค้ก	40,000
007	สถิตย์	50,000
069	พอลล่า	30,000

Teaches

fid	cid
001	204205
007	204203
007	204204
007	204329
069	202102
069	202103

Course

cid	cname	credits
202102	Information Technology I	3
202103	Information Technology II	3
204203	Software Design and Development	4
204204	Database Design and Development	4
204205	Multimedia Design and Development	4
204329	Web Technology	3

ผลลัพธ์

fid	fname	cid	Cname
001	แพนเค้ก	204205	Multimedia Design and Development
007	สถิตย์	204203	Software Design and Development
007	สถิตย์	204204	Database Design and Development
007	สถิตย์	204329	Web Technology
069	พอลล่า	202102	Information Technology I
069	พอลล่า	202103	Information Technology II

เราสามารถใช้คำสั่ง **JOIN** ในการแสดงข้อมูลในคำสั่ง SQL ได้ในทำนองเดียวกันกับการเชื่อมตารางสองตารางดังนี้

```
SELECT f.fid, fname, c.cid, cname
FROM (faculty f JOIN teaches t USING fid) AS ft JOIN Course c USING cid;
```

<u>หลักการทำงานของ JOIN</u>

วิธีการคำนวณบนฐานข้อมูลเชิงสัมพันธ์เพื่อให้ได้มาซึ่งผลลัพธ์ของการดำเนินการ JOIN นั้นอาศัยพื้นฐานของผลลัพธ์จากการดำเนินการหาผลคูณคาร์ทีเซียน จากนั้นอาศัยการดำเนินการอื่นๆ อีก กล่าวได้ว่าผลลัพธ์ที่เกิดจากการดำเนินการ JOIN เป็นซับเซตของผลลัพธ์ที่ได้จากการดำเนินการหาผลคูณคาร์ทีเซียน เราใช้ **CROSS JOIN** สำหรับดำเนินการหาผลคูณคาร์ทีเซียนของตารางด้วยคำสั่ง SQL ซึ่งมีรูปแบบคำสั่งดังนี้

```
SELECT [DISTINCT | ALL] {* | column_list}
FROM table_name1 CROSS JOIN table_name2
```

การใช้ **CROSS JOIN** ในรูปแบบที่ได้แสดงคือการ **SELECT** ข้อมูลจากตารางข้อมูลหลายๆ ตาราง โดยไม่มีการกรองด้วย **WHERE** จากรูปแบบการใช้ที่ได้แสดง เราสามารถใช้ร่วมกับ WHERE และ ORDER BY ได้เช่นเดียวกับการ JOIN โดยทั่วไป

ขั้นตอนการคำนวณของการดำเนินการ JOIN สามารถอธิบายได้ดังนี้

1) คำนวณหาผล**คูณคาร์ทีเซียน**จากตารางที่ระบุใน **FROM**
2) ในกรณีที่มีการใช้ **WHERE** ให้กรองเฉพาะทูเพิลที่ตรงตามเงื่อนไข ซึ่งนั่นคือการดำเนินการ **restriction** ในพีชคณิตเชิงสัมพันธ์
3) สำหรับแถวที่เหลืออยู่จากการดำเนินการ ทำการสร้างข้อมูลตามรายการใน **SELECT** และสร้างแถวทีละแถว
4) ถ้ามีการใช้ **SELECT DISTINCT** ให้กำจัด แถวที่ซ้ำกันออก การดำเนินการในข้อ 3 และ 4 คือ การดำเนินการ **projection** บน **selection**
5) เรียงลำดับตามคอลัมน์ที่กำหนดในกรณีมีการใช้ **ORDER BY**

<u>OUTER JOIN</u>

โดยปกติแล้วผลลัพธ์จากการ JOIN ระหว่างตารางจะแสดงเฉพาะข้อมูลที่สามารถเชื่อมกันได้ ในกรณีที่ทูเพิลใดๆ ในตารางทางซ้ายไม่สามารถเชื่อมกับทูเพิลใดๆ ในตารางทางขวาได้เลย ทูเพิลนั้นจะไม่ปรากฏในผลลัพธ์ ในทำนองเดียวกันกับทูเพิลทางขวาที่ไม่สามารถเชื่อมกับทูเพิลใดๆ ของตารางทางซ้าย ทูเพิลนั้นจะไม่ปรากฏในผลลัพธ์ เราเรียกการ JOIN ในลักษณะนี้ว่า INNER JOIN

พิจารณาตัวอย่างตารางข้อมูลนักศึกษาและตารางสาขาวิชา โดย did ในตารางนักศึกษาซึ่งได้แก่รหัสสาขาวิชาที่นักศึกษาสังกัด อ้างถึง did ซึ่งได้แก่รหัสประจำสาขาวิชาในตารางสาขาวิชา ซึ่งข้อมูลที่ยกตัวอย่างนี้สามารถเชื่อมกันได้ด้วย did โครงสร้างตารางนักศึกษาที่เพิ่มคอลัมน์ did นี้เป็นตัวอย่างประกอบสำหรับหัวข้อนี้เท่านั้น สำหรับคอลัมน์ institute เป็นคอลัมน์แสดงสำนักวิชาที่สาขาวิชานั้นๆ สังกัดอยู่ ในกรณีนี้ตาราง Department ยังไม่ได้อยู่ในรูปแบบบรรทัดฐานเนื่องจากสำนักวิชานั้นซ้ำๆ กัน หรือ Department อาจเป็น View ก็ได้

Student

cid	sid	name	login	age	gpa	*did*
3100904022132	B5075666	สมชาย	somchai@it	18	3.44	204
3100904032132	B5075688	สมศรี	somsri@com	18	3.21	408
3100905622132	B5075650	สมศรี	somsri@it	19	3.82	204
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80	103
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00	NULL

Department

did	dname	dphone	institute
D102	สาขาวิชาฟิสิกส์	4187	สำนักวิชาวิทยาศาสตร์
D204	สาขาวิชาเทคโนโลยีสารสนเทศ	4273	สำนักวิชาเทคโนโลยีสังคม
D205	สาขาวิชาเทคโนโลยีการจัดการ	4267	สำนักวิชาเทคโนโลยีสังคม
D423	สาขาวิชาวิศวกรรมคอมพิวเตอร์	4422	สำนักวิชาวิศวกรรมคอมพิวเตอร์
D609	สาขาวิชาแพทย์ศาสตร์	3917	สำนักวิชาแพทย์ศาสตร์

ตัวอย่างที่ 8.27 JOIN

จากข้อมูลตัวอย่าง เราสามารถแสดงข้อมูลนักศึกษาพร้อมกับชื่อสาขาวิชาที่นักศึกษาสังกัดได้ด้วย SQL และผลลัพธ์ดังต่อไปนี้

```
SELECT s.*, d.dname
FROM student s JOIN department d ON s.did = d.did
```

ผลลัพธ์

cid	sid	name	login	age	gpa	did	dname
3100904022132	B5075666	สมชาย	somchai@it	18	3.44	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904032132	B5075688	สมศรี	somsri@com	18	3.21	D423	สาขาวิชาวิศวกรรมคอมพิวเตอร์
3100905622132	B5075650	สมศรี	somsri@it	19	3.82	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80	D609	สาขาวิชาแพทย์ศาสตร์

จากตัวอย่างดังกล่าว ผลลัพธ์ที่ได้เป็นไปในลักษณะการดำเนินการ JOIN เช่นเดียวกับตัวอย่างอื่นๆ โดยผลลัพธ์ไม่ปรากฏระเบียนของนักศึกษา “สมน้ำหน้า” เนื่องจากนักศึกษาคนดังกล่าวยังไม่มีสาขาวิชาสังกัด นอกจากนี้สาขาวิชาบางสาขาวิชาที่ยังไม่มีนักศึกษาสังกัดก็ไม่ได้แสดงในผลลัพธ์ เช่นสาขาวิชาฟิสิกส์ เป็นต้น

ในบางกรณีผู้ใช้ข้อมูลมีความประสงค์ที่จะแสดงข้อมูลทั้งหมดในตารางทางซ้ายของการ JOIN แม้ว่าจะไม่มีข้อมูลทูเพิลใดๆ จากตารางทางขวาที่สามารถเชื่อมข้อมูลกันได้ เช่นต้องการแสดงข้อมูลของนักศึกษาทั้งหมด พร้อมทั้งชื่อสาขาวิชาที่สังกัด รวมทั้งนักศึกษาที่ยังไม่ได้สังกัดสาขาวิชาด้วย หรือมีความประสงค์จะแสดงข้อมูลจากตารางทางขวาทั้งหมดแม้จะไม่มีข้อมูลจากตารางทางซ้ายที่เชื่อมกันได้ เช่นต้องการแสดง

ข้อมูลของสาขาวิชาทั้งหมดพร้อมทั้งรายชื่อนักศึกษากรณีที่มีนักศึกษาสังกัด หรือแม้แต่ผู้ใช้มีความประสงค์แสดงข้อมูลจากทั้งสองตาราง โดยแสดงทั้งข้อมูลที่ทูเพิลสามารถเชื่อมกันได้ และไม่สามารถเชื่อมกันได้ทั้งจากตารางซ้ายและขวา ซึ่งได้แก่การดำเนินการ OUTER JOIN ในพีชคณิตเชิงสัมพันธ์ ซึ่งสามารถเขียนให้อยู่ในรูปของคำสั่ง SQL ได้ดังนี้

ตัวอย่างที่ 8.28 Left OUTER JOIN

แสดงข้อมูลนักศึกษาทั้งหมดพร้อมทั้งชื่อสาขาวิชาที่นักศึกษาคนนั้นๆ สังกัด รวมถึงแสดงข้อมูลนักศึกษาที่ไม่มีสังกัดสาขาวิชาด้วย

SELECT s.*, d.dname

FROM student s LEFT JOIN department d ON s.did = d.did

cid	sid	name	login	age	gpa	did	dname
3100904022132	B5075666	สมชาย	somchai@it	18	3.44	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904032132	B5075688	สมศรี	somsri@com	18	3.21	D423	สำนักวิชาวิศวกรรมคอมพิวเตอร์
3100905622132	B5075650	สมศรี	somsri@it	19	3.82	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80	D609	สาขาวิชาแพทย์ศาสตร์
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00	NULL	NULL

ผลลัพธ์ที่ได้จากการใช้ **LEFT JOIN** เป็นการแสดงข้อมูลของตารางทางซ้ายทั้งหมด จึงได้ชื่อว่า LEFT JOIN สำหรับนักศึกษา “สมน้ำหน้า” ที่ยังไม่มีสาขาวิชาสังกัดนั้น ผลลัพธ์ที่เกิดจากตารางทางขวา ซึ่งได้แก่ตาราง Department จะมีค่าเป็น NULL

ตัวอย่างที่ 8.29 Right OUTER JOIN

แสดงข้อมูลนักศึกษาทั้งหมดพร้อมทั้งชื่อสาขาวิชาที่นักศึกษาคนนั้นๆ สังกัด รวมถึงแสดงรายชื่อสาขาวิชาแม้สาขาวิชานั้นๆ จะไม่มีนักศึกษาในสังกัด

SELECT s.*, d.dname

FROM student s RIGHT JOIN department d ON s.did = d.did

cid	sid	name	login	age	gpa	did	dname
3100904022132	B5075666	สมชาย	somchai@it	18	3.44	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904032132	B5075688	สมศรี	somsri@com	18	3.21	D423	สำนักวิชาวิศวกรรมคอมพิวเตอร์
3100905622132	B5075650	สมศรี	somsri@it	19	3.82	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80	D609	สาขาวิชาแพทย์ศาสตร์
NULL	NULL	NULL	NULL	NULL	NULL	NULL	สาขาวิชาฟิสิกส์
NULL	NULL	NULL	NULL	NULL	NULL	NULL	สาขาวิชาเทคโนโลยีการจัดการ

ตัวอย่างที่ 8.30 Full OUTER JOIN

แสดงข้อมูลนักศึกษาทั้งหมดพร้อมทั้งชื่อสาขาวิชาที่นักศึกษาคนนั้นๆ สังกัด ทั้งนี้รวมถึงแสดงข้อมูลนักศึกษาที่ไม่มีสังกัดสาขาวิชาด้วย และรายชื่อสาขาวิชาแม้สาขาวิชานั้นๆ จะไม่มีนักศึกษาในสังกัด

SELECT s.*, d.dname

FROM student s FULL JOIN department d ON s.did = d.did

cid	sid	name	login	age	gpa	did	dname
3100904022132	B5075666	สมชาย	somchai@it	18	3.44	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904032132	B5075688	สมศรี	somsri@com	18	3.21	D423	สาขาวิชาวิศวกรรมคอมพิวเตอร์
3100905622132	B5075650	สมศรี	somsri@it	19	3.82	D204	สาขาวิชาเทคโนโลยีสารสนเทศ
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80	D609	สาขาวิชาแพทย์ศาสตร์
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	สาขาวิชาฟิสิกส์
NULL	NULL	NULL	NULL	NULL	NULL	NULL	สาขาวิชาเทคโนโลยีการจัดการ

8.2.8 มีอยู่ และ ไม่มีอยู่ (EXISTS and NOT EXISTS)

EXISTS ใช้กับข้อคำถามย่อยสำหรับการตรวจสอบเงื่อนไขการมีอยู่อย่างน้อย 1 ระเบียนข้อมูล โดยข้อคำถามย่อย EXISTS จะมีค่าเป็น**จริง** ก็ต่อเมื่อข้อคำถามย่อย SELECT สามารถแสดงผลลัพธ์ได้อย่างน้อย 1 ระเบียน และในทางตรงกันข้าม ข้อคำถามย่อย **NOT EXISTS** จะมีค่าเป็น**จริง** ก็ต่อเมื่อข้อคำถามย่อย SELECT ไม่สามารถแสดงผลลัพธ์ได้เลยแม้แต่ระเบียนเดียว

การตรวจสอบการมีอยู่อย่างน้อย 1 ระเบียนหรือไม่มีเลยนั้นตรวจสอบเฉพาะจำนวนระเบียน เราจึงใช้ SELECT * FROM ในข้อคำถามย่อย เนื่องจากไม่สนใจในจำนวนหรือค่าของคอลัมน์ใดๆ

ตัวอย่างที่ 8.31 EXISTS

แสดงข้อมูลนักศึกษาที่ลงทะเบียนเรียนรายวิชา 204204

```
SELECT sid, name
FROM Student s
WHERE EXISTS (SELECT *
        FROM enrolled e
        WHERE s.sid = e.sid AND cid = '204204');
```

คำสั่ง SQL ข้างต้นจะตรวจสอบข้อมูลของนักศึกษาที่ปรากฏในตาราง Enrolled และมีการลงทะเบียนเรียนวิชา 204204 ซึ่งหากพบ ข้อคำถามย่อยก็จะมีค่าเป็น**จริง** หรือ **true** สำหรับระเบียนของนักศึกษาแต่ละระเบียนในข้อคำถามหลักที่ทำการตรวจสอบ ในลักษณะของ SQL ดังนี้

```
SELECT sid, name
FROM Student s
WHERE true;
```

ระเบียนของนักศึกษาในข้อคำถามหลักคนใด ที่ข้อคำถามย่อย EXISTS มีค่าเป็น true ข้อมูลจะถูกแสดง ซึ่งการใช้คำสั่ง SQL ในลักษณะดังกล่าวให้ผลลัพธ์เช่นเดียวกันกับการเชื่อมตารางในรูปแบบของการ JOIN ดังนี้

```
SELECT sid, name
FROM Student s, enrolled e
WHERE s.sid = e.sid AND cid = '204204';
```

8.2.9 Combining Result Tables (UNION, INTERSCET, EXCEPT)

การใช้ UNION, INTERSECT และ EXCEPT เป็นการดำเนินการแบบเซต อาจเลือกใช้ในกรณีที่คำสั่ง SQL มีความซับซ้อน สามารถแยกคำสั่ง SQL ออกจากกันเป็น 2 ชุดและนำผลลัพธ์มารวมหรือมากระทำต่อกันต่อไป ตัวอย่างต่อไปนี้แสดงวิธีการใช้คำสั่ง UNION, INTERSECT และ EXCEPT อย่างง่าย

ตัวอย่างที่ 8.32 UNION

แสดงข้อมูลนักศึกษาที่ลงทะเบียนเรียนรายวิชา 204204 และ 204203

```
(SELECT sid, name, gpa
FROM Student
WHERE sid IN (SELECT sid
        FROM enrolled
        WHERE cid = '204203')
UNION
(SELECT sid, name, gpa
FROM Student
WHERE sid IN (SELECT sid
        FROM enrolled
        WHERE cid = '204204')
```

จากตัวอย่าง คำสั่ง SELECT ในส่วนแรกแสดงข้อมูลจากตารางนักศึกษา โดยที่ WHERE ทำการเปรียบเทียบ sid กับ sid ที่ได้จากข้อคำถามย่อยในการแสดงรหัสนักศึกษา sid จากตาราง enrolled ตามรหัสรายวิชาที่ระบุไว้ ซึ่งตัวอย่างเป็นการแสดงข้อมูลนักศึกษาที่ลงทะเบียนรายวิชา 2040204 จากนั้น นำมารวมกับผลลัพธ์ที่ได้ในการแสดงผลคำสั่ง SELECT อีกชุดหนึ่งหลัง UNION ผลลัพธ์ที่ได้คือข้อมูลนักศึกษาที่ลงทะเบียนในทั้ง 2 รายวิชา ให้สังเกตการใช้เครื่องหมายวงเล็บ "( )" ครอบชุดคำสั่ง SELECT ทั้ง 2 โดยให้ระลึกว่าผลลัพธ์ที่สามารถนำมา UNION กันได้ต้องเข้ากันได้ กล่าวคือมีจำนวนและลำดับของคอลัมน์เท่ากัน

ตัวอย่างที่ 8.33 INTERSECT

แสดงข้อมูลนักศึกษาที่ลงทะเบียนเรียนทั้งรายวิชา 204204 และ 204203

```
(SELECT sid, name, gpa
FROM Student
WHERE sid IN (SELECT sid
        FROM enrolled
        WHERE cid = '204203')
INTERSECT
(SELECT sid, name, gpa
FROM Student
WHERE sid IN (SELECT sid
        FROM enrolled
        WHERE cid = '204204')
```

จากตัวอย่างสามารถอธิบายได้ในทำนองเดียวกันกับตัวอย่างการใช้ UNION ต่างกันตรงที่ผลลัพธ์ที่ได้คือข้อมูลของนักศึกษาที่ลงทะเบียนทั้ง 2 รายวิชา นักศึกษาที่ลงทะเบียนรายวิชาใดวิชาหนึ่งเท่านั้นจะไม่ปรากฏในผลลัพธ์

ตัวอย่างที่ 8.34 EXCEPT

แสดงข้อมูลนักศึกษาที่ลงทะเบียนเรียนรายวิชา 204203 แต่ต้องไม่ลงทะเบียนเรียนรายวิชา 204204 ด้วย

```
(SELECT sid, name, gpa
FROM Student
WHERE sid IN (SELECT sid
        FROM enrolled
        WHERE cid = '204203')
EXCEPT
(SELECT sid, name, gpa
FROM Student
WHERE sid IN (SELECT sid
        FROM enrolled
        WHERE cid = '204204')
```

จากตัวอย่างสามารถอธิบายได้ในทำนองเดียวกันกับตัวอย่างการใช้ UNION และ INTERSECTION แต่การใช้ EXCEPT นั้นตรงตามความหมายของคำสั่งคือการยกเว้น โดยการแสดงผลลัพธ์จากคำสั่ง SELECT ในชุดแรก ยกเว้นระเบียนที่ปรากฏในผลลัพธ์ของ SELECT ในชุดที่สองซึ่งอยู่หลังจากคำสั่ง EXCEPT การใช้งาน EXCEPT นี้ตรงกันกับการดำเนินการ DIFFERENCE ในพีชคณิตเชิงสัมพันธ์

8.3 การเพิ่ม การแก้ไข และการลบข้อมูล (Insertion, Update and Deletion)

นอกจากคำสั่ง SELECT ที่ใช้ในการแสดงข้อมูลแล้ว คำสั่ง SQL ในกลุ่มของ DML ยังได้แก่คำสั่งที่ใช้ในการเพิ่ม การแก้ไข และการลบข้อมูล ซึ่งได้แก่คำสั่ง INSERT, UPDATE และ DELETE ตามลำดับ ทั้งนี้คำสั่ง SELECT ใช้สำหรับแสดงข้อมูลที่บรรจุอยู่ในฐานข้อมูลอยู่แล้ว และเป็นเพียงการแสดงข้อมูลเท่านั้น ข้อมูลที่บรรจุอยู่ในฐานข้อมูลจะไม่มีการเปลี่ยนแปลงใดๆ คำสั่ง SQL ในหัวข้อนี้เป็นคำสั่งที่ใช้ในการเปลี่ยนแปลงข้อมูลดังต่อไปนี้

8.3.1 การเพิ่มข้อมูล (INSERT)

การเพิ่มข้อมูลเข้าไปในตาราง ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

```
INSERT INTO <table_name> [(column_list)]
VALUES (data_value_list)
```

โดย

INSERT INTO คือคำสั่งที่ใช้ในการเพิ่มข้อมูล

table_name ชื่อของตารางที่ต้องการแทรกข้อมูลเข้าไป

column_list คือรายการของคอลัมน์ข้อมูลที่ต้องการเพิ่มข้อมูล ในกรณีที่เพิ่มข้อมูลหลายๆ คอลัมน์ ให้ใช้เครื่องหมายจุลภาค “,” คั่นระหว่างคอลัมน์

VALUES ประกอบคำสั่ง INSERT INTO เพื่อระบุว่าแทรกข้อมูลด้วย “ค่า” ที่กำหนด

data_value_list รายการของค่าข้อมูลที่ต้องการเพิ่ม ในกรณีที่เพิ่มข้อมูลหลายๆ คอลัมน์ ให้ใช้เครื่องหมายจุลภาค “,” ขั้นระหว่างค่า โดยค่าจะต้องสัมพันธ์กับ column_list หรือโครงสร้างของตารางข้อมูลที่จะทำการเพิ่มข้อมูล

สามารถเขียนรูปแบบคำสั่งเพิ่มข้อมูล ที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

เพิ่ม เข้าไปยัง <ชื่อตาราง> [(รายการของคอลัมน์)]
ด้วยค่า (รายการของค่า);

นอกจากนี้การแทรกค่าเข้าไปในตารางข้อมูลยังสามารถแทรกค่าได้โดยการนำค่าที่ได้จากคำสั่ง SELECT เพิ่มลงในฐานข้อมูลด้วยรูปแบบดังนี้

```
INSERT INTO <table_name> [(column_list)]
<select_statement>
```

โดย

select_statement คือคำสั่ง SQL ที่ใช้ในการสร้างข้อมูลจากตารางข้อมูลต่างๆ เพื่อนำมาแทรกในตารางที่กำหนดในคำสั่ง INSERT INTO

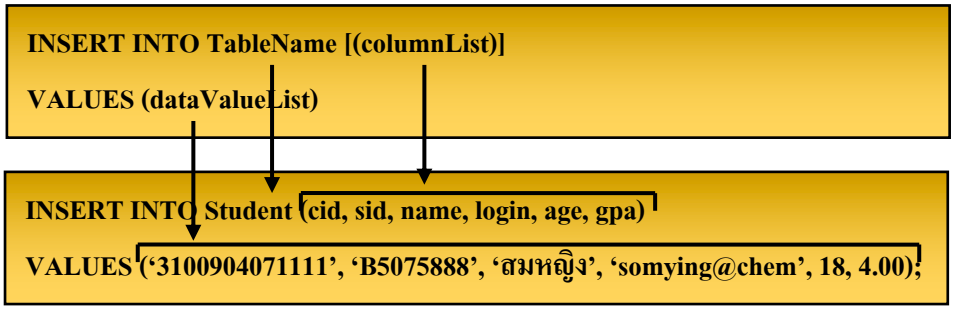
ตัวอย่างที่ 8.35 INSERT

เพิ่มข้อมูลเข้าไปในตารางนักศึกษา 1 ระเบียน โดยกำหนดค่าให้ทุกคอลัมน์

INSERT INTO Student (cid, sid, name, login, age, gpa)

VALUES ('3100904071111', 'B5075888', 'สมหญิง', 'somying@chem', 18, 4.00);

จากตัวอย่าง ส่วนของคำสั่ง **INSERT INTO Student** เป็นการกำหนดให้เพิ่มข้อมูลที่ระบุไว้ลงในตาราง Student สำหรับส่วน **(cid, sid, name, login, age, gpa)** นั้นเป็นการระบุรายการคอลัมน์ที่ต้องการเพิ่มข้อมูล หมายความว่าในบางครั้งตารางข้อมูลอาจมีจำนวนคอลัมน์มากกว่าที่ระบุก็ได้ แต่เราต้องการเพิ่มข้อมูลไม่ครบทุกคอลัมน์ ในตัวอย่างนี้เป็นการเพิ่มข้อมูลจำนวน 6 คอลัมน์ซึ่งเท่ากับจำนวนคอลัมน์ที่มีอยู่ในตาราง Student ในส่วนของ **VALUES ('3100904071111', 'B5075888', 'สมหญิง', 'somying@chem', 18, 4.00)** เป็นการระบุค่าที่ต้องการเพิ่ม แสดงได้ตามภาพต่อไปนี้



ผลลัพธ์ที่ได้จากการประมวลผลคำสั่งการเพิ่มข้อมูลดังตัวอย่างคือการเพิ่มข้อมูลลงในตาราง Student ด้วยระเบียนใหม่ 1 ระเบียน ดังนี้

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00
⇨ 3100904071111	B5075888	สมหญิง	somying@chem	18	4.00

ตัวอย่างที่ 8.36 INSERT

การเพิ่มข้อมูลโดยไม่ระบุรายการคอลัมน์ที่ต้องการเพิ่มข้อมูล

INSERT INTO Student

VALUES (‘3100904071111’, ‘B5 075888’, ‘สมหญิง’, ‘somying@chem’, 18, 4.00);

จากตัวอย่าง คำสั่ง INSERT INTO นี้ไม่มีรายการกำกับคอลัมน์ที่ต้องการเพิ่มข้อมูลในตารางหลังชื่อตารางซึ่งสามารถกระทำได้ในกรณีที่รายการของค่าที่จะแทรกมีจำนวนและลำดับตรงกันกับโครงสร้างของตารางที่เพิ่มข้อมูล แสดงได้ดังนี้

INSERT INTO Student

VALUES (‘3100904071111’, ‘B5075888’, ‘สมหญิง’, ‘somying@chem’, 18, 4.00);

Student

cid	Sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00
⇨ 3100904071111	B5075888	สมหญิง	somying@chem	18	4.00

การแทรกข้อมูลดังกล่าวอาจจะสะดวกแต่อาจก่อให้เกิดปัญหาขึ้นได้ภายหลัง เช่นในกรณีที่มีการแก้ไขสคีมาของตารางข้อมูลดังภาพต่อไปนี้

INSERT INTO Student

VALUES (‘3100904071111’, ‘B5075888’, ‘สมหญิง’, ‘somying@chem’, 18, 4.00);

Student

cid	Sid	name	login	did	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	D204	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	D423	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	D204	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	D609	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	NULL	22	2.00
⇨ 3100904071111	B5075888	สมหญิง	somying@chem	18	4.00	?

จากรูป ตารางข้อมูล Student อาจมีการเพิ่มคอลัมน์ did เพื่อระบุรหัสสาขาที่นักศึกษาแต่ละคนสังกัด คำสั่ง SQL ที่แสดงจึงไม่สามารถใช้ได้กับตารางดังกล่าว พิจารณาค่าในลำดับที่ 5 และ 6 ซึ่งได้แก่ค่า age และ gpa ตามสคีมาเดิมของตาราง ซึ่งคอลัมน์ทั้ง 2 ถูกเลื่อนลำดับไปด้วยคอลัมน์ did การเพิ่มข้อมูล “18” ลงในคอลัมน์ did และ “4.00” ลงในคอลัมน์ age จึงไม่ถูกต้อง ดังนั้นเราจึงควรระบุรายการของคอลัมน์ ไว้ในคำสั่ง INSERT ด้วย ซึ่งเป็นแนวปฏิบัติที่พึงกระทำ

ตัวอย่างที่ 8.37 INSERT

การเพิ่มข้อมูลโดยระบุรายการของคอลัมน์ไม่เรียงตามที่ปรากฏในสคีมาของตาราง

INSERT INTO Student (sid, cid, name, login, age, gpa)

VALUES (‘B5075888’, ‘3100904071111’, ‘สมหญิง’, ‘somying@chem’, 18, gpa);

จากตัวอย่าง ให้สังเกตรายการของคอลัมน์ ในลำดับที่ 1 และ 2 ของรายการในคำสั่ง INSERT INTO นั้นสลับกันกับลำดับที่ของคอลัมน์ในตาราง การกระทำเช่นนี้สามารถกระทำได้ แต่ค่าที่ระบุในรายการของค่าหลัง VALUES นั้นจะต้องสอดคล้องกันกับคอลัมน์ในรายการคอลัมน์ ในตัวอย่างค่า ‘B5075888’ ซึ่งเป็นค่าแรกในรายการเพิ่มข้อมูลสัมพันธ์กันกับคอลัมน์ sid และค่า ‘3100904071111’ ในลำดับถัดมาก็สัมพันธ์กันกับคอลัมน์ cid ทำให้เราสามารถใช้คำสั่งดังกล่าวในการเพิ่มข้อมูลได้อย่างถูกต้อง ซึ่งสามารถแสดงด้วยภาพได้ดังนี้

INSERT INTO Student (sid, cid, name, login, age)

VALUES (‘B5075888’, ‘3100904071111’, ‘สมหญิง’, ‘somying@chem’, 18);

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00
3100904071111	B5075888	สมหญิง	somying@chem	18	NULL

นอกจากนั้น ในตัวอย่างนี้ รายการของคอลัมน์มีเพียง 5 รายการเท่านั้น โดยคำสั่งไม่ได้ระบุค่าของ gpa ซึ่งสามารถกระทำได้ในกรณีที่เราอนุญาตให้ค่าในคอลัมน์ gpa สามารถเป็นค่าว่างได้ สำหรับในกรณีนี้หากไม่ได้กำหนดค่าโดยปริยายไว้ ค่า gpa จะมีค่าเป็นค่าว่างหรือ NULL

อย่างไรก็ตาม เราไม่สามารถละเว้นการกำหนดค่าในการเพิ่มข้อมูลสำหรับคอลัมน์ที่ไม่อนุญาตให้เป็นค่าว่างหรือได้กำหนดค่าโดยปริยายไว้ ดังนี้

INSERT INTO Student (name, login, age, gpa)

VALUES (‘สมหญิง’, ‘somying@chem’, 18, 4.00);

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00
NULL	NULL	สมหญิง	somying@chem	18	4.00

เนื่องจาก sid เป็น PRIMARY KEY ของตาราง เราจึงต้องระบุค่า sid ทุกครั้งในการเพิ่มข้อมูลในตาราง Student คำสั่ง INSERT INTO ที่ไม่ได้ระบุค่าของ sid นี้จึงไม่สามารถใช้งานได้

ตัวอย่างที่ 8.38 INSERT ด้วย SELECT

การเพิ่มข้อมูลโดยใช้ผลลัพธ์ที่ได้จากคำสั่ง SELECT

แทรกข้อมูลระเบียนใหม่ลงในตาราง student_hall_of_fame ด้วยข้อมูลของนักศึกษาที่มีเกรดเฉลี่ยสูงกว่า 3.25 และเป็นนักศึกษาที่รหัสนักศึกษาขึ้นต้นด้วย B50

INSERT INTO student_hall_of_fame (cid, sid, name, age, gpa)

SELECT cid, sid, name, age, gpa

FROM Student

WHERE gpa >= 3.25 AND sid LIKE ‘B50*’

จากตัวอย่าง ส่วนของคำสั่ง **INSERT INTO student_hall_of_fame (cid, sid, name, age, gpa)** เป็นการระบุตารางและคอลัมน์ข้อมูลที่จะทำการเพิ่มข้อมูล ซึ่งมีรูปแบบเหมือนคำสั่ง INSERT ทั่วๆ ไป สำหรับในส่วนที่ 2 นั้น แทนด้วยข้อคำถาม SELECT ซึ่งสามารถใช้คำสั่ง SELECT ได้ทุกรูปแบบ ผลลัพธ์ที่ได้ต้องสัมพันธ์กันกับรายการคอลัมน์ในการเพิ่มข้อมูล ในกรณีนี้ข้อคำถามเป็นการเลือกข้อมูลจากตาราง Student สำหรับนักศึกษาที่รหัสนักศึกษาขึ้นต้นด้วย B50 และมีเกรดเฉลี่ยมากกว่าหรือเท่ากับ 3.25 เพื่อคัดลอกข้อมูลของนักศึกษาเหล่านี้เข้าไปอยู่ในตาราง student_hall_of_fame ซึ่งตารางดังกล่าวเป็นตารางที่ใช้เก็บนักศึกษาที่จะได้รับการเชิดชูเกียรตินั่นเอง หากตาราง student_hall_of_fame นี้ยังไม่มีข้อมูลบรรจุอย่าง จะได้ผลลัพธ์ของตัวอย่างนี้ดังนี้

Student_Hall_Of_Fame

cid	sid	name	age	gpa
3100904022132	B5075666	สมชาย	18	3.44
3100905622132	B5075650	สมศรี	19	3.82

8.3.2 การแก้ไขข้อมูล (Data Updates)

การแก้ไข/ปรับข้อมูลในตาราง ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

```
UPDATE <table_name>
SET column_name1 = data_value1 [, column_name2 = data_value2…]
[WHERE search_condition]
```

โดย

UPDATE	คือคำสั่งที่ใช้ในการแก้ไขข้อมูล
table_name	ชื่อของตารางที่ต้องการแก้ข้อมูล
SET	ประกอบคำสั่ง UPDATE เพื่อระบุการ “กำหนด” ค่าตามที่ระบุ
column_name	คือชื่อของคอลัมน์ข้อมูลที่ต้องการแก้ไข
=	เครื่องหมายประกอบคำสั่งแก้ไขข้อมูล หมายถึงการกำหนดค่า
data_value	คือค่าใหม่ของข้อมูลที่ต้องการแก้ไข

คำสั่ง UPDATE หนึ่งคำสั่งสามารถแก้ไขข้อมูลพร้อมๆ กันได้ทีละหลายๆ คอลัมน์ โดยการใช้เครื่องหมายจุลภาค “,” ขั้นชุดของการกำหนดค่า โดยคำสั่ง UPDATE จะแก้ไขค่าคอลัมน์ที่กำหนดของทุกระเบียน แต่ถ้ามีการกำหนด search_condition ด้วย WHERE นั้น ระบบจัดการฐานข้อมูลจะแก้ไขเฉพาะระเบียนที่เป็นไปตามเงื่อนไขใน WHERE เท่านั้น

สามารถเขียนรูปแบบคำสั่งแก้ไขข้อมูล ที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

แก้ไข <ชื่อตาราง>

กำหนดให้ ชื่อคอลัมน์1 = ค่าใหม่1 [, ชื่อคอลัมน์2 = ค่าใหม่2]

[โดยที่ เงื่อนไข];

ตัวอย่างที่ 8.39 UPDATE

แก้ไขอายุนักศึกษาในตาราง Student โดยเปลี่ยนข้อมูลอายุของนักศึกษาที่มีอายุ 18 ปี ให้เป็นอายุ 19 ปี

UPDATE Student

SET age = 19

WHERE age = 18

จากตัวอย่าง ระบบจัดการฐานข้อมูลจะประมวลผลคำสั่งโดยการค้นหานักศึกษาที่มีอายุ 18 ปีตามเงื่อนไข **WHERE age = 18** ในตาราง Student จากนั้นจะทำการแก้ไขค่าในคอลัมน์ age ให้เป็น 19 ปี ตามส่วนของคำสั่ง **SET age = 19** แสดงได้ด้วยภาพต่อไปนี้

UPDATE Student SET age = 19

WHERE age = 18

Student

cid	Sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	**18**	3.44
3100904032132	B5075688	สมศรี	somsri@com	**18**	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

ได้ผลลัพธ์ดังนี้

Student

cid	Sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	19	3.44
3100904032132	B5075688	สมศรี	somsri@com	19	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	21	2.00

พึงระลึกว่าโดยตารางข้อมูลไม่ควรเก็บค่าในลักษณะอายุ แต่ควรจัดเก็บเป็นวันเดือนปีเกิด

ตัวอย่างที่ 8.40 UPDATE

แก้ไขอายุนักศึกษาในตาราง Student โดยเปลี่ยนข้อมูลของนักศึกษาที่มีรหัสนักศึกษาเท่ากับ B5075831 โดยแก้ไขอายุให้เพิ่มขึ้น 1 ปี และกำหนดให้ gpa มีค่าเท่ากับ 2.00

UPDATE Student SET age = age + 1, gpa = 2.00

WHERE sid = 'B5075831'

จากตัวอย่างเป็นการแก้ไขค่าของนักศึกษารหัส B5075831 จากเงื่อนไข **WHERE sid = 'B5075831'** โดยเป็นการแก้ไขค่าพร้อมๆ กัน 2 คอลัมน์ ได้แก่คอลัมน์ age และ gpa ซึ่งการแก้ไขค่านั้นเราสามารถกำหนดค่าใหม่ให้เป็นค่าที่ได้จากการคำนวณได้ เช่น **age = age + 1** ระบบจัดการฐานข้อมูลจะทำการค้นหานักศึกษารหัส B5075831 ก่อน จากนั้นจึงทำการแก้ไขข้อมูลคอลัมน์ age และ gpa ตามค่าที่กำหนดหรือค่าที่ได้จากการคำนวณ ซึ่งแสดงได้ด้วยภาพต่อไปนี้

UPDATE Student SET age = age + 1, gpa = 2.00

WHERE sid = 'B5075831'

Student

cid	Sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	**B5075831**	สมศักดิ์	somsak@med	**21**	**1.80**
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

ได้ผลลัพธ์ดังนี้

Student

cid	Sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	19	3.44
3100904032132	B5075688	สมศรี	somsri@com	19	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	22	2.00
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	21	2.00

8.3.3 การลบข้อมูล (Data Deletions)

การลบข้อมูลออกจากตาราง ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

DELETE FROM <table_name>

[WHERE search_condition]

โดย

DELETE FROM คือคำสั่งที่ใช้ในการลบข้อมูลออกจากตาราง

table_name ชื่อของตารางที่ต้องการลบระเบียนใดๆ

คำสั่ง DELETE จะทำการลบข้อมูลออกเป็นระเบียนๆ ซึ่งระเบียนใดที่ตรงกับเงื่อนไขที่กำหนดใน search_condition ด้วย WHERE จะถูกลบทิ้ง พึงสังเกตว่าคำสั่ง DELETE FROM นั้นไม่ได้ลบข้อมูลออกเป็นคอลัมน์

สามารถเขียนรูปแบบคำสั่งลบข้อมูล ที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

ลบ ออกจาก <ชื่อตาราง> [โดยที่ เงื่อนไข];

ตัวอย่างที่ 8.41

ลบข้อมูลของนักศึกษารหัส B5075831

DELETE FROM Student

WHERE sid = 'B5075831'

ระบบจัดการฐานข้อมูลจะทำการค้นหานักศึกษารหัส B5075831 จากนั้นจะทำการลบข้อมูลนักศึกษาคนดังกล่าวออกทั้งระเบียน

Student

cid	Sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
⇨ 3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

ได้ผลลัพธ์ดังนี้

cid	Sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	19	3.44
3100904032132	B5075688	สมศรี	somsri@com	19	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	21	2.00

การเพิ่ม แก้ไข และลบข้อมูล ต้องเป็นไปตามเงื่อนไขบังคับบูรณภาพของข้อมูลด้วย มิเช่นนั้นจะไม่สามารถดำเนินการตามคำสั่งได้

8.4 Stored Procedures and Triggers

Stored procedures และ triggers เป็นความสามารถของ SQL ที่ทำให้การใช้งานระบบจัดการฐานข้อมูลมีประสิทธิภาพมากขึ้น โดย stored procedures เป็นรูปแบบของคำสั่ง SQL ที่มีการทำงานเป็นขั้นตอนต่อเนื่อง ทำซ้ำ หรือเลือกทำได้เช่นเดียวกันกับภาษาคอมพิวเตอร์แบบ procedural language หรือภาษาเชิงกระบวนการ เช่นภาษาซี เป็นต้น แต่อยู่ในรูปของ SQL สำหรับ triggers นั้นเป็นความสามารถในการกำหนดให้ SQL ทำงานอย่างหนึ่งอย่างใดเมื่อมีเหตุการณ์บางอย่างเกิดขึ้น ทั้งในเอกสารเล่มนี้จะแนะนำ stored procedures และ triggers ในเบื้องต้นเพื่อให้เข้าใจแนวคิดและการใช้งานอย่างง่ายเท่านั้น การใช้งาน stored procedures และ triggers สามารถศึกษาได้ในหัวข้อชั้นสูงของระบบจัดการฐานข้อมูลต่อไป

8.4.1 Stored Procedures

คำสั่ง SQL จะมีประโยชน์เพิ่มขึ้นอีกมากหากเราสามารถนำคำสั่งทั้งกลุ่มของ DDL และ DML มาพัฒนาระบบโดยเขียนคำสั่งเป็นขั้นตอน มีกระบวนการ และโครงสร้าง เช่นเดียวกับคำสั่งเชิงกระบวนการ และโครงสร้างอื่นๆ แนวคิดของ stored procedures นี้ยังเกี่ยวข้องกับการจัดการธุรกรรมซึ่งจะกล่าวถึงในบทถัดไป และ triggers ในหัวข้อถัดไป

ในภาษาเชิงโครงสร้าง (structural language) เช่นภาษาซี เราสามารถรับค่าจากผู้ใช้ เปรียบเทียบค่าเพื่อเลือกทำ หรือทำซ้ำ เพื่อแสดงผลลัพธ์ (output) ที่ถูกต้องตามข้อมูลนำเข้า (input) ได้ ความสามารถในการรองรับการเขียนโปรแกรมเชิงกระบวนการและโครงสร้างใน SQL นี้ไม่ได้มีมาพร้อมกับ SQL ในรุ่นแรกๆ จะเป็นความสามารถพิเศษที่ผู้ผลิตระบบจัดการฐานข้อมูลแต่ละรายเพิ่มเข้ามาให้ได้เปรียบคู่แข่งและเพิ่มประสิทธิภาพการใช้งานฐานข้อมูล อีกทั้งการพัฒนาระบบฐานข้อมูลไม่ได้จำเป็นที่จะต้องเขียนคำสั่ง SQL เชิงกระบวนการ ทำให้มาตรฐานของคำสั่ง SQL เชิงกระบวนการนั้นถูกกำหนดได้ช้า syntax หรือไวยกรณ์ของ SQL เชิงกระบวนการของผู้ผลิตแต่ละรายในปัจจุบันจึงค่อนข้างหลากหลาย

ใน Oracle เราเรียกความสามารถเชิงกระบวนการของ SQL ว่า PL/SQL (Procedural Language/SQL) ใน Microsoft SQL Server เรียกว่า Transact-SQL (T-SQL) สำหรับใน MySQL นั้นเรียกว่า SQL/PSM (SQL/Procedural Stored Modules) ซึ่งตรงกับมาตรฐานที่กำหนดโดย ISO ในปี 2003—SQL:2003

เราสามารถใช้ SQL เชิงกระบวนการในการสร้างกระบวนการเพื่อรอบรับความต้องการของผู้ใช้ได้ตั้งแต่การดำเนินการกับข้อมูลง่ายๆ จนถึงระดับที่มีความซับซ้อนมากๆ ได้ ในบทที่ 10 ฐานข้อมูลบนเว็บได้มีการอธิบายถึงสถาปัตยกรรม 3-Tier ของซอฟต์แวร์ ซึ่งโดยปกติ ส่วนของโปรแกรมที่รองรับความต้องการของผู้ใช้จะแยกออกจากระดับฐานข้อมูล การใช้ SQL เชิงกระบวนการนั้นเป็นการฝังส่วนของโปรแกรมที่รองรับความต้องการของผู้ใช้ลงไปในฐานข้อมูลซึ่งอาจเพิ่มความเร็วในการทำงานของระบบแต่จะลดความเป็นอิสระระหว่างส่วนของแอพพลิเคชันกับฐานข้อมูลลง ซึ่งการพิจารณาใช้ SQL เชิงกระบวนการนั้นขึ้นอยู่กับความเหมาะสมในการใช้งาน

Stored procedures เป็นการเขียน SQL เชิงกระบวนการและจัดเก็บไว้เป็นโปรแกรมที่ทำงานบนระบบจัดการฐานข้อมูล stored procedures ที่สร้างขึ้นสามารถถูกเรียกใช้งานได้เรื่อยๆ นอกจาก stored procedures จะประมวลผลตามสถานะของข้อมูลในฐานข้อมูลขณะนั้นๆ แล้ว stored procedures ยังสามารถรับค่าที่ผู้ใช้

กำหนด รวมถึงสร้างผลลัพธ์หลากหลายรูปแบบได้ ในที่นี้จะแนะนำ stored procedure ในเบื้องต้นเท่านั้น โดยจะกล่าวถึงส่วนของคำสั่งในการสร้าง stored procedure เฉพาะเบื้องตนและตัวอย่างพื้นฐาน ซึ่งสามารถนำรูปแบบไปประยุกต์ใช้ได้ในโอกาสต่อไป การสร้าง stored procedures มีรูปแบบอย่างง่ายดังนี้

```
CREATE        [OR REPLACE] PROC [owner,] <procedure_name> [;number]
              [<@parameter_name> <datatype> [, …]]
[WITH {RECOMPILE | ENCRYPTION | RECOMPILE, ENCRYPTION}]
[FOR REPLICATION]
AS sql_statements;
```

โดย

CREATE OR REPLACE PROC	เป็นคำสั่งที่ใช้ในการสร้าง stored procedure ในกรณีที่ระบุ OR REPLACE ด้วย ระบบจัดการฐานข้อมูลจะแทนที่ stored procedure ที่มีอยู่เดิมในกรณีที่ชื่อซ้ำกัน
<procedure_name>	คือชื่อของ stored procedure
<@parameter_name> <data_type>	คือชื่อและชนิดของของพารามิเตอร์ ทั้งนี้ stored procedures สามารถรับค่าพารามิเตอร์จากผู้ใช้ได้เพื่อประกอบการประมวลผล ดังแสดงในตัวอย่างต่อไป
AS	ประกอบคำสั่ง CREATE PROC เพื่อระบุว่าสร้าง PROC "ด้วย" SQL เชิงกระบวนการที่กำหนด
sql_statements	คำสั่ง SQL ซึ่งอาจจะเป็น SQL ง่ายๆ หรือ SQL เชิงกระบวนการเต็มรูปแบบ

ตัวอย่างที่ 8.42 Triggers

สร้าง stored procedures สำหรับแสดงข้อมูลของนักศึกษาที่มีเกรดเฉลี่ยมากกว่าหรือเท่ากับ 3.25

```
CREATE PROC sp_show_gpa1
AS
SELECT sid, name, gpa
FROM Student
WHERE gpa >= 3.25;
```

จากตัวอย่าง เป็นการสร้าง stored procedure โดยใช้ชื่อว่า sp_show_gpa1 ด้วยคำสั่ง SQL ที่แสดงข้อมูล sid, name และ gpa ของนักศึกษาที่มีเกรดเฉลี่ยมากกว่า 3.25 การสร้าง stored procedure นั้นเป็นการสร้างวัตถุสำหรับประมวลผลเก็บไว้ในฐานข้อมูลเท่านั้นแต่ยังไม่ได้มีประมวลผลตามคำสั่งใน stored procedure จนกว่าจะมีการเรียกใช้งานจริง ซึ่งการเรียกใช้งาน stored procedure นั้นต้องใช้คำสั่ง EXEC เพื่อเรียกใช้งาน ตามด้วยชื่อของ stored procedure ดังนี้

EXEC sp_show_gpa1;

ได้ผลลัพธ์ดังนี้

Student

sid	name	gpa
B5075666	สมชาย	3.44
B5075650	สมศรี	3.82

ตัวอย่างที่ 8.43 Stored Procedures

สร้าง stored procedure ที่สามารถรับพารามิเตอร์ได้ เป็น stored procedure สำหรับแสดงข้อมูลของนักศึกษาที่มีเกรดเฉลี่ยมากกว่าหรือเท่ากับค่าพารามิเตอร์ที่ระบุ

CREATE PROC sp_show_gpa2

@gpa FLOAT

AS

SELECT sid, name, gpa

FROM Student

WHERE gpa >= @gpa;

จากตัวอย่างเป็นการสร้าง stored procedures โดยใช้ชื่อว่า sp_show_gpa2 ในบรรทัดถัดมาหลังจากชื่อของ stored procedure เป็นการประกาศรายการของพารามิเตอร์ที่สามารถนำไปใช้ในการประมวลผลซึ่งสามารถมีได้หลายพารามิเตอร์ ในตัวอย่างนี้ stored procedures รับพารามิเตอร์เพียงพารามิเตอร์เดียว ชื่อว่า @gpa มีชนิดข้อมูลเป็น FLOAT ชื่อของพารามิเตอร์จะต้องใช้เครื่องหมาย “@” นำหน้าเสมอ ทั้งในขณะประกาศพารามิเตอร์ และการอ้างถึงในขณะใช้ค่าพารามิเตอร์ใน stored procedures

Stored procedures ดังกล่าวใช้ในการแสดงข้อมูลของนักศึกษา ประกอบด้วย sid, name, gpa ดังเช่นตัวอย่างที่แล้ว แต่ในตัวอย่างนี้ จะแสดงข้อมูลของนักศึกษาที่มีเกรดเฉลี่ยมากกว่าหรือเท่ากับค่าที่กำหนดเป็นพารามิเตอร์ขณะเรียกใช้ stored procedures นี้ จากส่วนของคำสั่ง **WHERE gpa >= @gpa** เห็นได้ว่า เป็นการเปรียบเทียบค่า gpa กับค่าพารามิเตอร์ @gpa การเรียกใช้ stored procedures ให้ใช้คำสั่ง EXEC ตามด้วยชื่อ stored procedures เช่นเดียวกับการเรียกใช้งาน stored procedures โดยทั่วไป พร้อมทั้งระบุค่าพารามิเตอร์หลังจากชื่อของ stored procedures ดังนี้

EXEC sp_show_gpa2 3.25;

เป็นการแสดงข้อมูลของนักศึกษาที่มีเกรดเฉลี่ยมากกว่าหรือเท่ากับ 3.25 ซึ่งผลลัพธ์ที่ได้เป็นตารางข้อมูลเหมือนกันกับตัวอย่างที่แล้ว

ตัวอย่างอื่นๆ

EXEC sp_show_gpa2 3.50;

EXEC sp_show_gpa2 2.00;

EXEC sp_show_gpa2 1.80;

แสดงตัวอย่างการเรียกใช้ stored procedures sp_show_gap2 ในการแสดงข้อมูลนักศึกษาที่มีเกรดเฉลี่ยมากกว่าหรือเท่ากับ 3.50, 2.00 และ 1.80 ตามลำดับ

เราสามารถดัดแปลงการสร้าง stored procedures ให้สามารถรับค่าพารามิเตอร์มากกว่า 1 ค่า พร้อมทั้งใช้งานได้อย่างถูกต้อง เพียงประกาศตัวแปรเพิ่มในรายการพารามิเตอร์ และกำหนดค่าพารามิเตอร์เพิ่มเมื่อเรียกใช้งาน stored procedures โดยใช้เครื่องหมายจุลภาค “,” ขั้นระหว่างชุดพารามิเตอร์ นอกจากนี้ stored procedures ยังมีความซับซ้อนได้มากกว่านี้มาก อาจมีการใช้คำสั่งควบคุม IF…ELSE ฯลฯ

8.4.2 Triggers

Trigger เป็นกลไกของระบบจัดการฐานข้อมูลที่เพิ่มเข้ามาเพื่อช่วยให้ระบบจัดการฐานข้อมูลทำงานบางอย่างโดยอัตโนมัติเมื่อเกิดเหตุการณ์ที่เรากำหนดไว้ได้ เช่น ให้นำข้อมูลเก่าการการเปลี่ยนแปลงไปจัดเก็บไว้ในตารางสำรองข้อมูล หรือให้ตรวจสอบค่าต่างๆ เป็นต้น ความสามารถที่ trigger ทำได้อาจเป็นในระดับง่ายๆ ไม่ซับซ้อน โดยอาศัย SQL ที่ไม่ซับซ้อน จนถึงการทำงานที่ซับซ้อนโดยการใช้ SQL เชิงกระบวนการและการใช้ stored procedures

เราอาจใช้ trigger ในการกำหนดเงื่อนไขบังคับบูรณภาพที่ซับซ้อนก็ได้ แต่เราจะหลีกเลี่ยงการใช้ trigger สำหรับการดำเนินการดังกล่าวถ้า CHECK CONSTRAINTS สามารถตรวจสอบได้เนื่องจากการจัดการ CONSTRAINTS ที่มากับระบบจัดการฐานข้อมูลมีประสิทธิภาพมากกว่า

รูปแบบของคำสั่ง trigger และการใช้งานรวมกับ SQL เชิงกระบวนการรวมถึง stored procedures นั้นแตกต่างกันไปพอสมควรตามระบบจัดการฐานข้อมูลของผู้ผลิตแต่ละราย ในที่นี้อธิบายการใช้งาน trigger ที่อิงมาตรฐาน SQL:2003 เป็นส่วนใหญ่ โดยจะได้ยกตัวอย่างการใช้งาน trigger สำหรับสร้างตารางตรวจสอบเส้นทาง (audit trail) ที่เป็นการดำเนินการอันหนึ่งที่เรามักนิยมใช้ประโยชน์จาก trigger

การสร้าง trigger ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

```
CREATE          [OR REPLACE] TRIGGER <trigger_name>
{BEFORE|AFTER}  {INSERT|DELETE|UPDATE} ON <table_name>
FOR EACH ROW
<trigger_statement>
```

โดย

CREATE OR REPLACE TRIGGER เป็นคำสั่งที่ใช้ในการสร้าง trigger ในกรณีที่ระบุ OR REPLACE ด้วย ระบบจัดการฐานข้อมูลจะแทนที่ trigger ที่มีอยู่เดิมในกรณีที่ชื่อซ้ำกัน

<procedure_name>	คือชื่อของ trigger
BEFORE|AFTER	ผู้สร้าง trigger สามารถระบุได้ว่าจะให้ trigger ทำงานก่อนและ/หรือหลังจากการเกิดเหตุการณ์ใดๆ
INSERT|DELETE|UPDATE	ระบุเหตุการณ์หรือการดำเนินการใดๆ ที่กำหนดให้ trigger ทำงาน
ON	ประกอบคำสั่ง CREATE TRIGGER เพื่อระบุว่าสร้าง TRIGGER "บน" ตารางใด
FOR EACH ROW	ประกอบคำสั่ง CREAT TIRGGER เพื่อระบุว่าสร้าง TRIGGER "สำหรับแต่ละแถว"
sql_statements	คำสั่ง SQL ซึ่งอาจจะเป็น SQL ง่ายๆ หรือ SQL เชิงกระบวนการเต็มรูปแบบ

ตัวอย่างที่ 8.44 Triggers

สร้าง stored procedures สำหรับแสดงข้อมูลของนักศึกษาที่มีเกรดเฉลี่ยมากกว่าหรือเท่ากับ 3.25

```
CREATE TRIGGER tr_student_audit_insert
ON Student
BEFORE INSERT
FOR EACH ROW
BEGIN
        INSERT INTO student_audit (sid, action, user, timestamp, gpa)
        VALUES (NEW.sid, 'I', CURRENT_USER(), NOW(), NEW.gpa)
END;
```

จากตัวอย่าง เป็นการสร้าง trigger โดยใช้ชื่อว่า tr_student_audit_insert ด้วยคำสั่ง SQL ที่ทำงานบนตาราง Student โดยจะทำงานทุกครั้งก่อนการ INSERT ข้อมูลลงในตาราง การทำงานของ trigger เรียกว่าการ fire trigger ซึ่งสิ่งที่ trigger ที่ได้ยกตัวอย่างดำเนินการได้แก่การเพิ่มระเบียนลงในตาราง student_audit ซึ่งเป็นตารางตรวจสอบการเปลี่ยนแปลงข้อมูลของตาราง Student ด้วยข้อมูล ดังนี้

sid	คือรหัสนักศึกษา ในที่นี้กำหนดค่าเท่ากับ NEW.sid ซึ่ง NEW ในที่นี้คือการระบุถึงค่าใหม่พร้อมกำกับด้วยชื่อคอลัมน์ โดยที่ trigger สำหรับการดำเนินการ UPDATE นั้นจะมีค่าทั้ง OLD และ NEW ที่สามารถอ้างอิงได้ ในขณะที่ trigger สำหรับการดำเนินการ DELETE นั้นมีเพียงค่า OLD ที่สามารถอ้างอิงได้
action	คือกิจกรรมหรือการดำเนินการที่ทำให้ trigger ทำงาน ในที่นี้ระบุเป็นค่าคงที่ 'I' หมายถึง INSERT

user ชื่อของผู้ใช้ฐานข้อมูลขณะนั้นๆ โดยปกติระบบสารสนเทศใดๆ จะกำหนดชื่อผู้ใช้ฐานข้อมูลโดยเฉพาะ ในที่นี้ใช้ค่าที่เรียกจากฟังก์ชัน CURRENT_USER() ซึ่งจะแต่งต่างกันออกไปสำหรับแต่ละระบบจัดการฐานข้อมูล การจัดการผู้ใช้กล่าวถึงเพิ่งเติมในบทที่ 9

timestamp เวลาที่ trigger นั้นๆ ทำงาน เพื่อใช้ในการตรวจสอบลำดับเหตุการณ์ที่เกิดขึ้น

gpa ค่าเกรดเฉลี่ยที่เพิ่มเข้ามาในฐานข้อมูล โดยใช้ค่า NEW.gpa หมายถึงค่า gpa ที่เพิ่งเพิ่มเข้ามาในฐานข้อมูล เราเก็บค่า gpa ไว้ตรวจสอบเนื่องจาก gpa เป็นข้อมูลที่มีความสำคัญ และเพื่อตรวจสอบเจตนาของการแก้ไขข้อมูลที่อาจเกิดจากการบุกรุกระบบได้

สำหรับ **BEGIN** และ **END** นั้นยกตัวอย่างเพื่อแนะนำการเขียน SQL ในรูปแบบของธุรกรรม กรณีที่มีชุดของ SQL หลายคำสั่ง ซึ่งเป็น syntax ของ SQL เชิงกระบวนการ หมายถึงเริ่มและสิ้นสุดชุดกระบวนการ

หลังจากการสร้าง trigger แล้วเสร็จ ระบบจะไม่ได้ดำเนินการอื่นใดนอกจากสร้างวัตถุ trigger ไว้ในฐานข้อมูล ซึ่งวัตถุ trigger นี้จะทำงานก็ต่อเมื่อเกิดเหตุการณ์หรือกิจกรรมตามที่กำหนดไว้ ในกรณีนี้ trigger tr_student_audit_insert จะทำงานเมื่อมีการเรียกใช้คำสั่ง INSERT กับตาราง Student ดังตัวอย่างต่อไปนี้จึงจะทำให้ trigger student_audit_insert ทำงาน

INSERT INTO Student (cid, sid, name, login, age, gpa)
VALUES (‘3100904078132, ‘B5075832, ‘สมน้ำหน้า’, ‘somnamna@math’, 21, 2.00)

เราสามารถสร้าง trigger สำหรับกิจกรรม UPDATE และ DELTE บนตาราง Student ได้อีก อาจสร้างแยกเป็น trigger ต่างหาก หรือสร้างรวมกันแล้วตรวจสอบว่า trigger นั้นเกิดจากกิจกรรมใด ตารางต่อไปนี้เป็นตัวอย่างของข้อมูลตารางตรวจสอบ student_audit หลังจากการดำเนินการกับตาราง Student ด้วยการดำเนินการมาแล้วระยะหนึ่ง

Student_Audit

audit_id	sid	action	user	timestamp	gpa
234	B5075688	U	enrollment	11/11/08 2:22:22 AM	3.21
235	B5075650	D	enrollment	11/11/08 3:33:33 PM	3.82
236	B5075832	U	enrollment	11/11/08 4:44:44 PM	2.00
⇨ 237	B5075832	I	enrollment	11/11/08 1:11:11 AM	2.00

คำสั่ง INSERT INTO Student ที่ได้ยกตัวอย่างสำหรับการทำให้ trigger tr_student_audit_insert ดำเนินการนั้น นอกจากจะทำให้ข้อมูลใหม่เพิ่มลงในตาราง Student แล้ว ยังมีข้อมูลสำหรับการตรวจสอบเพิ่มขึ้นในระเบียนสุดท้ายของตาราง student_audit ซึ่งมี audit_id เท่ากับ 237 ดังแสดง คอลัมน์ audit_id เป็น

PRIMARY KEY ที่ค่าสามารถเพิ่มเองได้โดยอัตโนมัติ ตาราง student_audit ต้องถูกสร้างขึ้นมาก่อนที่ trigger จะทำงาน

ตาราง student_audit นี้เป็นตารางที่ใช้ในการบันทึกกิจกรรมที่เกิดขึ้นกับตาราง Student เพื่อประโยชน์ในการสำรองข้อมูล การตรวจสอบความผิดพลาดของระบบแอพพลิเคชันว่ามีการคำนวณเกรดผิดพลาดหรือมีตรรกะของโปรแกรมผิดพลาดหรือไม่ และประโยชน์ประการสำคัญคือการตรวจสอบการแก้ไขที่ไม่ได้รับอนุญาต เป็นต้น เราเรียกตารางการตรวจสอบลักษณะนี้ว่าตารางตรวจสอบ (audit table) สำหรับการตรวจสอบเส้นทาง (audit trail) หรือตรวจสอบกิจกรรมใดๆ มันนิยมสร้างขึ้นเพื่อรองรับตารางข้อมูลที่มีความสำคัญ

8.5 บทสรุป

SELECT เป็นคำสั่ง SQL ในกลุ่มภาษาจัดการข้อมูล ซึ่ง SELECT นี้เป็นคำสั่งที่ใช้งานเป็นอย่างมาก เนื่องจากใช้ในการแสดงข้อมูลนั่นเอง เราใช้ WHERE ประกอบกับคำสั่ง SELECT เพื่อกำหนดเงื่อนไขที่จะแสดงเฉพาะข้อมูลระเบียนที่เป็นไปตามเงื่อนไข นอกจากนี้เรายังใช้คำสั่งอื่นๆ ประกอบ SELECT เพื่อกำหนดผลลัพธ์ได้อีก ได้แก่ ORDER BY ใช้สำหรับเรียงลำดับข้อมูล GROUP BY ใช้สำหรับจัดกลุ่มข้อมูลในการดำเนินการเกี่ยวกับฟังก์ชันคำนวณหรือสถิติต่างๆ เช่น SUM, AVG, MAX และ MIN ที่ใช้ในการหาผลรวม ค่าเฉลี่ย ค่าสูงสุดและค่าต่ำสุด เป็นต้น เราสามารถสร้างข้อคำถามย่อย SELECT ซ้อนคำสั่ง SELECT ได้ โดยนำผลลัพธ์ของคำสั่งย่อยไปใช้ในคำสั่งหลักของ SELECT

UNION ใช้ดำเนินการรวมตารางเข้าด้วยกัน โดยระเบียนที่เหมือนกันจะคงไว้เพียง 1 ระเบียน INSERTSECTION ใช้สำหรับรวมผลลัพธ์ของตารางเช่นกันแต่เลือกแสดงเฉพาะระเบียนที่เหมือนกัน ส่วน EXCEPT ใช้แสดงระเบียนที่มีในตารางหนึ่งแต่ไม่มีในอีกตารางหนึ่ง ซึ่งจะเห็นว่าคำสั่ง SQL มีการดำเนินการกับตารางข้อมูลเช่นเดียวกันกับพีชคณิตเชิงสัมพันธ์

INSERT, UPDATE และ DELETE เป็นคำสั่งในกลุ่มภาษาจัดการข้อมูลเช่นเดียวกัน ใช้สำหรับแทรกระเบียน ปรับปรุงข้อมูล และลบระเบียน ตามลำดับ คำสั่ง INSERT สามารถใช้ประกอบคำสั่ง SELECT ในการเลือกข้อมูลชุดหนึ่งมาเพิ่มลงในตารางฐานข้อมูล สำหรับการปรับ และลบระเบียน สามารถระบุระเบียนที่จะดำเนินการได้อย่างเฉพาะเจาะจงโดยการใช้ WHERE clause เช่นเดียวกันกับใน SELECT

SQL สามารถนำมาเขียนในรูปแบบของชุดคำสั่งขนาดใหญ่ดังเช่นภาษาคอมพิวเตอร์ยุคที่ 3 โดยทั่วไปได้ แต่เน้นที่การจัดการกับฐานข้อมูล ซึ่งเราเรียกว่าการเขียน SQL แบบ procedural จัดเก็บไว้ในรูปของ stored procedures นอกจากนี้เรายังสามารถสร้าง triggers ซึ่งเป็นชุดของคำสั่ง SQL ที่จะดำเนินการเมื่อเกิดเหตุการณ์การดำเนินการกับฐานข้อมูลตามที่ระบุไว้ได้ เราสามารถนำ SQL นี้ไปเป็นภาษาย่อยร่วมกับคอมพิวเตอร์ภาษาหลัก (host language) อื่น ในการพัฒนาโปรแกรมประยุกต์เพื่อใช้งานสร้างระบบสารสนเทศอย่างเต็มรูปแบบต่อไป

8.6 แบบฝึกหัดท้ายบท

1. ให้เขียนคำสั่ง SQL สำหรับ queries แสดงข้อมูลจากฐานข้อมูล “จอง-ฉัน-สิ” ในแบบฝึกหัดท้ายบทที่ 6 ข้อ 2.1-2.7
2. ให้เขียนคำสั่ง SQL เพื่อเพิ่มระเบียนข้อมูลต่อไปนี้ลงในฐานข้อมูล library จากแบบฝึกหัดท้ายบทที่ 7 ข้อ 1

member

	member_id	name	lastname	address	postal	phone
(A)	00001	สมคิด	อยู่เสมอ	222 ต.โคกกรวด อ.เมือง จ.นครราชสีมา	30000	0-4421-4124
(B)	00002	สมหญิง	งามทรัพย์	14/77 ถนนลาดพร้าว บางกะปิ กรุงเทพฯ	10300	0-2666-7766
(C)	00003	สมชาย	หล่อล้ำ	111/100 แขวงวังใหม่ เขตวังเก่า กรุงเทพฯ	10290	0-1111-2323
(D)	00004	สมปอง	มีกิจ	1/1 ถนนมุขมนตรี อ.เมือง จ.นครราชสีมา	30000	0-4444-4441
(E)	00004	สมศรี	งามทรัพย์	14/77 ถนนลาดพร้าว บางกะปิ กรุงเทพฯ	10290	0-1111-2323

book

	book_id	name	author	price
(F)	00001	สู้แล้วรวย	ทีมงาน “สู้แล้วรวย”	165.00
(G)	00002	ออกกำลังอย่างไรให้เหมาะกับรูปร่าง	พร ใหม่เอี่ยม	200.00
(H)	00003	ฐานข้อมูลเลือด	โชคสถิตย์ สอาดดี	200.00
(I)	00004	Microsoft Access ภาคพิสดาร	ส. ไม่ใหญ่	300.00

rental

	member_id	book_id	rent_date	return_date
(J)	00001	00001	10 MAY 2003	24 MAY 2003
(K)	00001	00001	27 MAY 2003	28 MAY 2003
(L)	00001	00002		27 AUG 2003
(M)	00001	00003	27 SEP 2003	
(N)	00003	00004	27 AUG 2003	
(O)	00001	00001	10 MAY 2003	
(P)	00005	00004	27 FEB 2003	27 MAR 2003

ในกรณีที่ไม่สามารถเพิ่มข้อมูลในฐานข้อมูลได้ ให้ระบุระเบียนพร้อมเหตุผลของการไม่สามารถเพิ่มข้อมูล

3. ให้นักศึกษาเขียนคำสั่ง SQL เพื่อรองรับ Transaction Requirements ข้อ 1, 4, 8 และ 9 ของโครงงาน “เช่า-ฉัน-สิ” ในภาคผนวก

4. ให้ออกแบบรายงาน 3 รายงานสำหรับช่วยในการบริหารจัดการธุรกิจ “เช่า-ฉัน-สิ” ในภาคผนวก จากนั้นให้สร้าง SQL สำหรับ queries ที่ใช้เพื่อแสดงข้อมูลในรายงานนั้น โดยใช้ relational model ที่ได้ออกแบบไว้ในแบบฝึกหัดท้ายบทที่ 4

บรรณานุกรมและเอกสารอ้างอิงประจำบท

ชนวัฒน์ ศรีสอ้าน. (2542). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

พงษ์พันธ์ ศิวิลัย. (2549). **SQL Server 2005 ฉบับสมบูรณ์.** กรุงเทพฯ : ซีเอ็ดยูเคชั่น.

ศุภกฤษฎิ์ นิวัฒนากูล. (2547). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

Connolly, T. and Begg, C. (2004). **Database Systems: A Practical Approach to Design, Implementation and Management, 4th ed.** Boston : Addison-Wesley.

Elmarsi, R. and Navathe, S. B. (2006). **Fundamentals of Database Systems, 5th ed.** Boston : Addison-Wesley.

Ramakrishnan, R. and Gehrke, J. (2003). **Database Management Systems, 3rd ed**. Singapore : McGraw-Hill.